

Universidad: pendiente completar nombre institucional oficial

Programa doctoral: pendiente completar denominacion formal

Version del documento: borrador de revision interna doctoral

A New Parallelization Approach in Deep Learning Using CPU/GPU Architectures for Memory Optimization

Monografia doctoral de investigacion

Tesis sobre optimizacion de memoria y ejecucion heterogenea en
entrenamiento profundo

Doctorando: Luis Alejandro Torres

Direccion: pendiente completar director(a) y codirector(a)

Lugar y fecha de presentacion: pendiente completar ciudad, pais y mes de
deposito

Fecha de compilacion: 26 de marzo de 2026

Resumen

Esta monografía estudia el problema de la partición heterogénea CPU-GPU para entrenamiento profundo como un problema metodológico y experimental completo, no solo como una heurística de asignación. La tesis sostiene que una política de partición solo adquiere valor científico cuando puede fundarse en profiling empírico reproducible, traducirse a una formulación declarativa con restricciones físicas explícitas y cerrarse en simulación y ejecución observada sobre runtime real. Sobre esa base, el manuscrito construye un pipeline en cinco fases que enlaza medición por capa, agregación robusta, optimización ILP, representación formal del plan y validación híbrida con trazabilidad de artefactos.

Los resultados disponibles muestran que el proyecto ya dispone de cierre metodológico funcional: mide, decide, simula y ejecuta. El profiling produce coeficientes auditables de tiempo, energía, memoria y transferencia; el ILP encuentra políticas superiores a baselines simples en casos controlados y campañas smoke; y el runtime híbrido permite contrastar el comportamiento previsto frente al observado, incluyendo evidencia focal de extensiones avanzadas como separación forward/backward y prefetching. La principal brecha restante no es de implementación sino de amplitud empírica: falta consolidar la campaña multi-servidor y ampliar la matriz final de validación para convertir la evidencia focal actual en cierre doctoral transversal.

Palabras clave: partición heterogénea CPU-GPU; entrenamiento profundo; optimización ILP; profiling empírico; gestión de memoria; simulación y runtime híbrido.

Abstract

This dissertation studies heterogeneous CPU-GPU partitioning for deep learning training as a full methodological and experimental problem rather than as an isolated placement heuristic. The central claim is that a partitioning policy only becomes scientifically meaningful when it is grounded on reproducible empirical profiling, translated into a declarative optimization model with explicit physical constraints, and finally validated through simulation and observed hybrid execution on a real runtime. Accordingly, the manuscript develops a five-phase pipeline that links per-layer measurement, robust aggregation, ILP optimization, formal plan representation, and runtime validation under a traceable evidence contract.

Current results already demonstrate methodological closure: the project can measure, decide, simulate, and execute. Profiling yields auditable coefficients for time, energy, memory, and transfer; the ILP improves upon simple baselines in smoke and controlled settings; and the hybrid runtime supports prediction-versus-observation analysis, including focused evidence for advanced features such as forward/backward separation and explicit prefetching. The main remaining gap is therefore empirical breadth rather than software capability: a final multi-server campaign is still required to transform the current focal evidence into a fully doctoral-grade cross-platform validation.

Keywords: heterogeneous CPU-GPU partitioning; deep learning training; integer linear programming; empirical profiling; memory management; hybrid runtime validation.

Convenciones del manuscrito

El documento distingue con rigor entre capacidad implementada, validación focal y cierre empírico amplio. Cuando una afirmación describe una capacidad del sistema ya materializada en el repositorio, se formula en presente. Cuando una afirmación depende de evidencia controlada disponible pero todavía no generalizada a una matriz experimental completa, se explicita su alcance focal. Cuando una conclusión requiere una campaña final aun no ejecutada, se marca como trabajo de cierre pendiente y no como resultado ya demostrado. Esta convención evita inflar el alcance de la evidencia y mantiene la coherencia epistemológica del manuscrito.

Alcance de esta version

La presente version debe leerse como un borrador doctoral avanzado de preentrega interna. Su estructura argumental, su cadena metodologica y su soporte empirico principal ya estan integrados, pero todavia quedan por completar ciertos elementos propios de la version de deposito: datos institucionales definitivos de portada, consolidacion de la campana multi-servidor, ampliacion bibliografica comparada y ajuste final de algunos cuadros y figuras. Esta explicacion no debilita el manuscrito; delimita con precision el alcance de lo ya demostrado y preserva la honestidad epistemologica del documento. En particular, los campos institucionales marcados en rojo no responden a una carencia de estructura del manuscrito, sino a la ausencia de informacion oficial cerrada dentro del repositorio de trabajo.

Siglas y abreviaturas

CPU	Unidad Central de Procesamiento.
GPU	Unidad de Procesamiento Grafico.
ILP	Programacion Lineal Entera.
VRAM	Memoria de video residente en el acelerador.
DAG	Grafo aciclico dirigido.
DDP	Paralelismo de datos distribuido.
FSDP	Paralelizacion completamente fragmentada de parametros y estados.

Índice general

Resumen	i
Abstract	ii
Convenciones del manuscrito	iii
Alcance de esta version	iv
Siglas y abreviaturas	v
1. Introduccion, Problema y Diseno de la Investigacion	1
1.1. 1.1 Contexto e introduccion	1
1.2. 1.2 Motivacion tecnica y relevancia cientifica	3
1.3. 1.3 Planteamiento formal del problema	5
1.4. 1.4 Hipotesis de investigacion	6
1.5. 1.5 Objetivos general y especificos	7
1.6. 1.6 Alcance decisional fijado (resultado de Fase 0)	8
1.7. 1.7 Metodologia general y validacion	9
1.8. 1.8 Contribucion cientifica	10
1.9. 1.9 Organizacion del documento	12
2. Marco Teorico y Fundamentacion del Problema	13
2.1. 2.1 Entrenamiento de redes profundas y grafos computacionales	13
2.2. 2.2 Arquitecturas heterogeneas CPU-GPU y jerarquias de memoria	14
2.3. 2.3 Transferencia de datos, solapamiento y comunicacion	16
2.4. 2.4 Precision numerica, estabilidad y costo material del entrenamiento	17
2.5. 2.5 Estrategias de gestion de memoria	18
2.6. 2.6 Paradigmas contemporaneos de paralelizacion y sus limites	19
2.7. 2.7 Optimizacion declarativa: ILP en sistemas computacionales	21

2.8.	2.8 Sintesis del marco teorico y delimitacion del vacio metodologico	23
2.9.	2.9 Estado del arte comparativo en clave problematica	24
3.	Arquitectura Metodologica del Sistema	27
3.1.	3.1 Vision general del pipeline experimental	27
3.2.	3.2 Estructura modular del software	28
3.3.	3.3 Contrato de artefactos y trazabilidad	29
3.4.	3.4 Definicion de plan y su representacion	30
3.5.	3.5 Criterios de validez y riesgos metodologicos iniciales	31
4.	Profiling Empirico para Particion Heterogenea CPU-GPU	33
4.1.	4.1 Motivacion empirica y preguntas de medicion	33
4.2.	4.2 Arquitectura de instrumentacion	33
4.3.	4.3 Modelo de energia y carga computacional	34
4.4.	4.4 Politica de precision y validez	34
4.5.	4.5 Extraccion estructural del grafo	35
4.6.	4.6 Modelo de transferencia consciente de arista	35
4.7.	4.7 Agregacion estadistica robusta	36
4.8.	4.8 Validez experimental y reproducibilidad	36
4.9.	4.9 Del dato observado al coeficiente de decision	38
5.	Formulacion ILP Robusta para Particion Heterogenea CPU-GPU	40
5.1.	5.1 Introduccion y alcance del modelo (congelado en Fase 0)	40
5.2.	5.2 Notacion formal	41
5.3.	5.3 Variables de decision	41
5.4.	5.4 Funcion objetivo multicriterio	42
5.5.	5.5 Linealizaciones exactas y restricciones	42
5.6.	5.6 Robustificacion estadistica	43
5.7.	5.7 Integracion multi-hardware	44
5.8.	5.8 Complejidad y trazabilidad	44
5.9.	5.9 Supuestos de modelado y frontera de aplicabilidad	45
5.10.	5.10 Validacion comparativa contra baselines (habilitada por Fase 1)	45
5.11.	5.11 Estudios de sensibilidad (habilitados por Fase 1)	46
5.12.	5.12 Ablaciones formales (habilitadas por Fase 1)	47
5.13.	5.13 Extension avanzada: persistencia de activaciones y rematerializacion (Fase 4)	48

5.14.	5.14 Amenazas a la validez	48
6.	Ejecucion Hibrida, Simulacion y Validacion Experimental	50
6.1.	6.1 Introduccion: cierre del bucle planificacion-ejecucion	50
6.2.	6.2 Representacion formal del plan de ejecucion (Fase 2)	51
6.3.	6.3 Simulador hibrido: arquitectura y validacion topologica (Fase 2)	51
6.4.	6.4 Estudio de prediccion: latencia, memoria y costos de transferencia (Fase 2)	51
6.5.	6.5 Implementacion del ejecutor hibrido en PyTorch (Fase 3)	52
6.6.	6.6 Baselines reales y comparativos experimentales (Fase 3)	53
6.7.	6.7 Prediccion frente a observacion: contraste operacional (Fase 3)	53
6.8.	6.8 Exactitud final del modelo y estabilidad de convergencia (Fase 3-5) . .	55
6.9.	6.9 Extensiones con persistencia de activaciones y scheduling asincrono (Fase 4)	56
6.10.	6.10 Analisis integrado de resultados y limites experimentales (Fase 5) . . .	56
6.11.	6.11 Significado metodologico del cierre predictivo	57
7.	Conclusiones, Limites y Trabajo Futuro	58
7.1.	7.1 Sintesis de resultados frente a la hipotesis	58
7.2.	7.2 Contribuciones metodologicas y de sistemas	58
7.3.	7.3 Limites del modelo y del protocolo experimental	59
7.4.	7.4 Lineas de trabajo futuro	60
7.5.	7.5 Trazabilidad final entre afirmaciones y evidencia	61
7.6.	7.6 Valor doctoral del trabajo en su estado actual	61
7.7.	7.7 Cierre final	62

Índice de cuadros

2.1. Dimensiones teóricas que estructuran el problema de partición heterogénea.	17
2.2. Paradigmas de paralelizacion y razon de insuficiencia frente al problema doctoral.	20
2.3. Comparacion conceptual entre familias de soluciones existentes y la orientacion de esta tesis.	22
3.1. Correspondencia entre etapa metodologica y tipo de evidencia producida. .	30
4.1. Contrato minimo de artefactos del pipeline de profiling.	38
4.2. Extracto empirico de coeficientes robustos de profiling para <code>simple_mlp</code> . .	38
4.3. Cadena de transformacion desde observacion cruda hasta coeficiente util para optimizacion.	39
5.1. Mejor resultado ILP factible por modelo bajo los presupuestos GPU evaluados en el conjunto smoke.	46
5.2. Barrido Pareto ILP en el conjunto smoke con comparacion frente a baselines.	47
5.3. Amenazas a la validez del modelo ILP y estado de mitigacion.	49
6.1. Comparacion prediccion-observacion en el caso dual controlado de <code>simple_mlp</code>	54
7.1. Trazabilidad entre afirmaciones nucleares del manuscrito y evidencia disponible.	61

Índice de figuras

3.1.	Figura conceptual del flujo de transformacion de evidencia en la tesis. . . .	28
4.1.	Perfil empirico real por capa para <code>simple_mlp</code> en smoke con batch 8 y precision <code>fp32</code> . La figura se genera directamente a partir de <code>simple_mlp_metrics_stats.csv</code>	37
5.1.	Frontera de compromiso objetivo-presupuesto GPU para el caso smoke consolidado de <code>simple_mlp</code>	46
6.1.	Contraste empirico entre simulacion y ejecucion hibrida observada para el caso dual controlado de <code>simple_mlp</code> . La figura se genera directamente a partir de los resumentes JSON de simulacion y runtime.	55
7.1.	Mejora porcentual del mejor plan ILP frente al baseline <code>all_cpu</code> en la campana de Fase 1. Esta figura resume la direccion favorable de la evidencia ILP antes del cierre empirico multi-servidor.	59

Capítulo 1

Introduccion, Problema y Diseno de la Investigacion

1.1. 1.1 Contexto e introduccion

El entrenamiento de redes neuronales profundas ha incrementado de forma sostenida su complejidad computacional, su densidad paramétrica y sus demandas de memoria, hasta el punto de convertir la disponibilidad de recursos hardware en una condición metodológica del propio resultado científico. En la práctica contemporánea del aprendizaje profundo, la GPU concentra la mayor parte de la carga crítica por su capacidad de cómputo paralelo, mientras que la CPU queda frecuentemente relegada a tareas auxiliares. Sin embargo, esta distribución de funciones no es una ley natural del entrenamiento, sino una convención operativa que responde más a la simplicidad de implementación que a una optimización rigurosa del sistema completo.

La consecuencia inmediata de esa convención es bien conocida: la memoria de la GPU se convierte en el cuello de botella dominante durante el entrenamiento de modelos profundos, especialmente cuando se incrementan resolución, longitud de secuencia, número de capas o tamaño del lote. Al mismo tiempo, la CPU y su espacio de memoria principal permanecen infrautilizados como recurso estructural del proceso de entrenamiento. Esta asimetría no solo limita la escalabilidad, sino que introduce una pregunta de investigación legítima: si la carga del entrenamiento puede distribuirse de forma científicamente controlada entre CPU y GPU, entonces la política de colocación deja de ser un detalle de ingeniería y pasa a constituir un objeto formal de estudio. La literatura sobre rematerialización, checkpointing y particion automatica confirma que esta tension entre memoria, computo y comunicacion es estructural y no contingente [2, 1, 5, 4].

Desde una perspectiva abstracta, el entrenamiento puede entenderse como un proceso de minimización de una función de pérdida sobre un espacio paramétrico:

$$\theta^* = \arg \min_{\theta} \mathcal{L}(\theta; \mathcal{D}),$$

pero esa formulación, por sí sola, oculta una dimensión material decisiva: la trayectoria hacia θ^* depende de una realización física concreta del grafo de entrenamiento. La tesis parte precisamente de esta observación. La optimización estadística del modelo y la organización computacional que la hace posible no son problemas independientes, sino dos caras de un mismo proceso. Por ello, la disponibilidad y la asignación de recursos dejan de ser cuestiones puramente instrumentales y pasan a integrarse dentro del razonamiento científico del trabajo.

La presente monografía se sitúa precisamente en ese punto. Su problema rector no es meramente acelerar código ni describir un conjunto de scripts, sino formular una teoría operacional de la partición heterogénea CPU-GPU para entrenamiento profundo bajo restricciones físicas explícitas. El trabajo entiende la heterogeneidad como un espacio de decisión sobre el cual es posible razonar, medir, optimizar y validar. En consecuencia, la tesis adopta una perspectiva de sistemas y de optimización experimental: mide el comportamiento por capa, robustifica coeficientes, construye un modelo declarativo, transforma la solución en un plan ejecutable y contrasta ese plan con evidencia observada.

Desde esta perspectiva, el valor científico del problema reside en que obliga a unir tres dominios que con frecuencia aparecen separados: la fenomenología empírica del hardware, la formalización matemática de la decisión y la verificación operacional de la política resultante. Si cualquiera de estos planos se omite, el resultado queda metodológicamente incompleto. Una optimización sin datos observados se vuelve abstracta; un profiling sin modelo de decisión es descriptivo pero no normativo; y una solución sin ejecución verificable no alcanza estatuto suficiente de evidencia doctoral en una tesis de sistemas.

En consecuencia, la introducción del problema no puede reducirse a una narrativa de escasez de memoria. Lo que está en juego es la posibilidad de construir una teoría aplicada del entrenamiento heterogéneo donde el diseño del sistema deje de ser un conjunto de decisiones implícitas y se convierta en una política formalmente justificable. Esa transición, desde la intuición ingenieril hacia la decisión explícita bajo restricciones, es el punto de partida conceptual de toda la monografía.

Esta formulación inicial cumple además una función de encuadre epistemológico. Permite desplazar el foco desde la pregunta instrumental de si una técnica concreta acelera o no una corrida hacia una pregunta más fuerte: bajo que condiciones una redistribución

de computo puede considerarse una decision cientificamente defendible. La diferencia no es retorica. Una mejora puntual de rendimiento puede ser valiosa en ingenieria aplicada; una tesis doctoral, en cambio, debe explicar por que esa mejora es interpretable, reproducible y susceptible de generalizacion conceptual.

En este sentido, la introduccion no opera solo como entrada narrativa al manuscrito, sino como delimitacion del tipo de conocimiento que se busca producir. La tesis no pretende ofrecer una receta local de optimizacion para un modelo particular, sino una forma de pensar la ejecucion heterogenea cuando el hardware deja de ser un soporte transparente y se convierte en parte constitutiva del problema. Esta orientacion es la que justifica el paso desde el analisis descriptivo del entrenamiento hacia una arquitectura metodologica que combina observacion, formalizacion y contraste operacional.

1.2. 1.2 Motivacion tecnica y relevancia cientifica

La motivación técnica del trabajo nace de una tensión concreta entre dos hechos empíricos. El primero es que el entrenamiento profundo depende de la retención temporal de activaciones, gradientes y estados del optimizador, lo cual produce una presión acumulativa sobre la memoria de la GPU. El segundo es que la CPU coexiste en la misma plataforma con recursos de memoria y cómputo significativos, pero rara vez participa de manera estructurada en la ejecución del grafo de entrenamiento. La tesis parte de la hipótesis de que esta asimetría no expresa un límite arquitectónico inevitable, sino la ausencia de un marco formal que permita explotar la heterogeneidad sin romper la coherencia operacional del entrenamiento.

Ese marco no puede construirse mediante reubicaciones ad hoc de capas ni mediante intuiciones locales basadas solo en throughput. Requiere una visión integrada de cuatro elementos: costos nodales por capa, dependencias estructurales del grafo, penalización de transferencias entre dispositivos y restricciones de memoria sobre ambos subsistemas. En otras palabras, la CPU no puede incorporarse como recurso complementario útil si la tesis no modela simultáneamente latencia, energía, memoria y comunicación. La dificultad técnica del problema es precisamente que toda decisión local de colocación modifica el equilibrio global del sistema. Ese diagnostico tambien aparece, con diferentes lenguajes, en trabajos sobre paralelizacion de pipeline, particion automatica y optimizacion de colocacion, donde la calidad de la decision depende de dependencias globales y no de reglas locales [3, 6, 11].

Si se denota de manera esquemática el costo total del entrenamiento como una combi-

nación de cómputo, memoria y comunicación,

$$Z \sim Z_{comp} + Z_{mem} + Z_{comm},$$

entonces la motivación técnica del trabajo puede formularse como la búsqueda de políticas que no minimicen un único término de forma miope, sino que controlen el compromiso entre los tres. Un desplazamiento agresivo hacia GPU puede reducir Z_{comp} y, a la vez, volver inviable Z_{mem} ; un reparto excesivo hacia CPU puede aliviar memoria pero inflar Z_{comm} y degradar el régimen temporal del entrenamiento. La tesis adquiere sentido precisamente porque este compromiso no puede resolverse por intuición local, sino mediante un esquema explícito de decisión.

La relevancia científica surge de convertir ese problema práctico en una metodología reproducible y defendible. La tesis no solo pregunta si la CPU puede participar más, sino bajo qué régimen de restricciones, con qué costo de transferencia, con qué efecto sobre la memoria disponible y con qué consecuencias sobre la calidad final del entrenamiento. En este punto el trabajo se aleja deliberadamente de un enfoque de benchmarking convencional: su meta no es producir una tabla de tiempos, sino un marco de inferencia experimental que permita derivar políticas de asignación interpretables y auditables.

En términos más amplios, el proyecto se inscribe en la transición hacia un aprendizaje profundo consciente de recursos. Esta transición supone que la eficiencia ya no puede definirse exclusivamente por latencia o throughput, sino por la capacidad de sostener entrenamiento útil bajo limitaciones materiales reales. Por ello, la tesis se sitúa en la intersección entre sistemas heterogéneos, optimización discreta y metodología experimental, y adopta como criterio de calidad no la mera novedad algorítmica, sino la coherencia entre medición, modelado y validación.

La relevancia científica del tema reside además en su capacidad de generalización conceptual. Aunque el caso de estudio se desarrolla sobre entrenamiento profundo CPU-GPU, la pregunta subyacente es más amplia: cómo convertir restricciones materiales del sistema en variables explícitas de decisión dentro de un marco formal. En este sentido, la tesis no solo aspira a producir una solución técnica utilizable, sino también a proponer un modo de razonar sobre aprendizaje automático cuando el recurso físico deja de ser abundante y pasa a formar parte del problema mismo.

Esa capacidad de generalización es particularmente importante en el contexto contemporáneo del aprendizaje profundo. A medida que los modelos crecen, la distancia entre formulación estadística y realización material se vuelve más estrecha. Ya no basta con demostrar que un método converge o que una arquitectura mejora una métrica final; es

necesario mostrar también que tal comportamiento puede sostenerse bajo restricciones realistas de memoria, energía y comunicación. La motivación científica del trabajo se asienta precisamente en esa mutación del campo: la eficiencia de sistemas deja de ser un apéndice ingenieril y pasa a formar parte del contenido cognitivo de la investigación.

Por ello, la tesis se coloca deliberadamente en una frontera interdisciplinar. No pertenece únicamente al análisis de sistemas heterogéneos ni exclusivamente al aprendizaje automático, sino al espacio en que ambos dominios se exigen mutuamente coherencia. Esta posición hace más exigente la argumentación del manuscrito, pero también la vuelve más valiosa: permite que cada resultado pueda leerse a la vez como proposición sobre entrenamiento profundo y como proposición sobre organización material del cómputo.

1.3. 1.3 Planteamiento formal del problema

El entrenamiento de redes neuronales profundas sigue una secuencia bien definida de operaciones distribuidas entre las fases de propagación hacia adelante (forward) y propagación hacia atrás (backward). Durante el backward, es imprescindible conservar las activaciones intermedias generadas por cada capa, ya que son necesarias para el cálculo de gradientes mediante la regla de la cadena. Este requisito impone una demanda acumulativa y sostenida sobre el subsistema de memoria, particularmente en arquitecturas de redes neuronales que son entrenadas en Unidades de Procesamiento Gráfico (GPU).

En las implementaciones estándar, la totalidad del grafo computacional se asigna a un único dispositivo GPU, bajo la suposición de que su alto rendimiento en paralelo puede compensar las limitaciones físicas de capacidad de memoria. Sin embargo, la memoria disponible en la GPU es fija, jerárquicamente segmentada y no ampliable mediante abstracciones de software. Esta restricción se vuelve crítica al entrenar modelos profundos que procesan entradas de alta resolución o mantienen representaciones internas densas, como en redes tipo Transformers o en canalizaciones convolucionales 3D.

Las técnicas convencionales, incluyendo checkpointing, compresión de tensores, aritmética de baja precisión y rematerialización selectiva, alivian parcialmente la presión sobre la memoria, pero actúan exclusivamente dentro del dominio del acelerador. De forma análoga, los paradigmas de entrenamiento distribuido ofrecen escalabilidad en múltiples nodos GPU, pero no resuelven la saturación de memoria en configuraciones monodispositivo o en entornos con recursos locales limitados.

Por su parte, las Unidades Centrales de Procesamiento (CPU), que coexisten en la misma plataforma física, disponen de capacidades computacionales complementarias,

espacios de memoria direccionables más amplios y soporte para ejecución vectorizada. A pesar de estas ventajas, permanecen subutilizadas durante el entrenamiento, no por insuficiencia arquitectónica, sino por la ausencia de un marco formal que permita integrarlas en la asignación de cargas bajo criterios consistentes de rendimiento y viabilidad.

Desde la perspectiva de la optimización discreta, este escenario constituye un problema de asignación por capas en un sistema heterogéneo, donde cada operación debe mapearse a un dispositivo de cómputo sujeto a restricciones observables: capacidad de memoria, latencia de transferencia, coste energético relativo y viabilidad de recomputación de activaciones, streaming o prefetching. El espacio de soluciones debe preservar la semántica de ejecución, respetar las dependencias del grafo y mantener la viabilidad global del sistema.

En consecuencia, la presente investigación aborda la siguiente brecha: la ausencia de un modelo de optimización formalmente definido, resoluble y parametrizado con métricas empíricas, que permita distribuir de forma eficiente las cargas de entrenamiento de los modelos de aprendizaje profundo entre CPU y GPU, minimizando la presión sobre la memoria de los aceleradores y preservando la viabilidad de ejecución, la eficiencia en el entrenamiento y la precisión del modelo en plataformas heterogéneas.

1.4. 1.4 Hipotesis de investigacion

El entrenamiento supervisado en redes neuronales profundas implica la ejecución secuencial de operaciones tensoriales a través de múltiples etapas computacionales. Durante la fase de retropropagación, es necesario conservar en memoria las activaciones intermedias para calcular correctamente los gradientes mediante la regla de la cadena. Esta dependencia temporal genera una carga acumulativa sobre el subsistema de memoria de la GPU, cuya capacidad es físicamente limitada, jerárquicamente segmentada y no ampliable mediante software. Tales restricciones condicionan el volumen de activaciones que pueden almacenarse simultáneamente, comprometiendo la viabilidad del entrenamiento en modelos con alta densidad computacional o arquitecturas jerárquicas profundas, donde el almacenamiento intermedio se convierte en un recurso crítico.

En respuesta a este fenómeno, la hipótesis central de esta investigación plantea que es posible reducir la presión sobre la memoria de la GPU durante el entrenamiento de modelos de aprendizaje profundo mediante la distribución estructural de operaciones entre los subsistemas CPU y GPU, formalizada a través de un modelo de optimización discreta por capas, parametrizado con métricas reales de ejecución y aplicable a entornos heterogéneos,

sin comprometer la exactitud del modelo ni la viabilidad de ejecución. Esta hipótesis implica que el entrenamiento de modelos de aprendizaje profundo puede reformularse como un problema de asignación computacional con restricciones, basado en recursos físicos medibles y verificables; que técnicas como la rematerialización y la transmisión interdispositivo pueden integrarse como decisiones internas dentro de un marco declarativo de optimización; y que el modelo resultante permite explorar configuraciones de ejecución no triviales, en las que la CPU actúa como un componente activo del grafo computacional, compensando las limitaciones de la GPU mediante la distribución inteligente de operaciones.

La validación de esta hipótesis se llevará a cabo mediante la construcción de un modelo de Programación Lineal Entera (ILP) que formalice la asignación óptima de operaciones, la simulación de su comportamiento sobre grafos de entrenamiento representativos y la comparación experimental de sus resultados frente a esquemas tradicionales centrados en GPU, evaluando indicadores como el uso máximo y promedio de memoria, la penalización en tiempo de ejecución, el consumo energético y la exactitud final del entrenamiento.

1.5. 1.5 Objetivos general y específicos

El objetivo general de esta investigación es formular, implementar y validar empíricamente una estrategia estructurada de paralelización para el entrenamiento de redes neuronales profundas, que permita la distribución a nivel de capa de las cargas computacionales entre subsistemas CPU y GPU con el fin de reducir la presión sobre la memoria de los aceleradores y, simultáneamente, preservar la precisión del modelo, la viabilidad de ejecución y la eficiencia energética, bajo restricciones arquitectónicas realistas y métricas de rendimiento observables, aportando un marco metodológico reproducible para el análisis y la optimización del entrenamiento sobre entornos de cómputo heterogéneo.

Los objetivos específicos son los siguientes.

1. Caracterizar empíricamente el uso de memoria y el comportamiento de ejecución de arquitecturas de redes profundas representativas, identificando factores arquitectónicos y operativos que contribuyen a la saturación de la memoria de la GPU, mediante perfiles obtenidos en entornos reales de CPU y GPU.
2. Formalizar la asignación computacional por capas como un modelo de optimización discreta, incorporando variables de decisión para la colocación en los dispositivos, la persistencia de activaciones, las transferencias entre dispositivos y las estrategias

de gestión de memoria, como la rematerialización, el streaming asíncrono y, cuando aplique, el prefetching, sujetas a restricciones de recursos y dependencias del grafo.

3. Parametrizar el modelo de optimización utilizando métricas observadas empíricamente, tales como huella de memoria por capa, latencia de ejecución, consumo energético y limitaciones específicas de cada dispositivo, garantizando que la formulación refleje dinámicas reales de entrenamiento.
4. Evaluar escenarios de ejecución híbrida CPU-GPU aplicando el modelo ILP a grafos de entrenamiento representativos, analizando el impacto de las decisiones de ubicación y recomputación sobre el uso de la memoria, el rendimiento temporal y la eficiencia de la asignación para el ajuste del modelo.
5. Validar experimentalmente el marco propuesto cuantificando el ahorro de memoria, las métricas de rendimiento, la eficiencia energética y la precisión final del modelo, e identificando las condiciones prácticas bajo las cuales la participación de la CPU mejora la eficiencia del entrenamiento frente a estrategias tradicionales centradas en la GPU.

1.6. 1.6 Alcance decisional fijado (resultado de Fase 0)

El alcance decisional del trabajo fue fijado explícitamente para evitar una deriva reductionista de la contribución. La tesis no se compromete con una simple asignación estática única por capa, sino con una formulación fuerte que contempla asignación independiente para *forward* y *backward*, persistencia de activaciones como variable explícita del modelo y compatibilidad con estrategias de transferencia asíncrona y *prefetching*. Esta decisión de alcance es metodológicamente importante porque impide confundir un prototipo parcial con la versión científica comprometida por la investigación.

La consecuencia epistemológica de este cierre es directa. El manuscrito puede utilizar formulaciones base y casos piloto como escalones explicativos, pero no puede presentar esas versiones intermedias como si agotaran la propuesta doctoral. Todo el texto debe leerse a la luz del contrato fuerte: la tesis aspira a una política de partición que no solo elija dispositivo por nodo, sino que gobierne también aspectos de disponibilidad de activaciones y sincronización entre subsistemas.

Desde esta perspectiva, el alcance no es un apartado administrativo, sino un mecanismo de disciplina científica. Fijar lo que la tesis realmente promete evita inflar aportes, ordena la validación por fases y mantiene coherencia entre escritura, software y evidencia. Esta coherencia es especialmente necesaria en un proyecto donde la complejidad técnica podría inducir a presentar como cierre doctoral lo que en realidad sería solo una capacidad exploratoria aún no sometida a contraste suficiente.

1.7. 1.7 Metodologia general y validacion

La investigación se desarrolla a partir de un enfoque mixto que combina modelado formal y validación experimental, orientado a optimizar el entrenamiento de redes neuronales profundas bajo restricciones arquitectónicas verificables. El problema se formula de manera declarativa, representando las operaciones del grafo de entrenamiento como entidades discretas asignadas a CPU o GPU, sujetas a limitaciones de memoria, latencia, consumo energético y viabilidad estructural. Cada etapa metodológica se concibe como una instancia formalizable y verificable, sustentada en un modelo computacional explícito, mecanismos de trazabilidad algorítmica y criterios de validación reproducibles, garantizando consistencia y rigor científico.

El marco metodológico se estructura en cinco fases secuenciales e interdependientes. En la primera fase, se caracteriza empíricamente el comportamiento de cargas de trabajo de varios modelos de deep learning en plataformas físicas con CPU y GPU, identificando factores arquitectónicos y operativos que inciden en la saturación de memoria de la GPU. Para ello, se seleccionan arquitecturas representativas por su complejidad secuencial, densidad computacional y sensibilidad a la gestión de activaciones. Las ejecuciones reales se instrumentan con herramientas de perfilado y las métricas obtenidas, uso activo de memoria, duración de cómputo, tamaño de activaciones, latencias de transferencia, consumo energético y limitaciones específicas, se organizan en vectores temporales y matrices de observabilidad que constituyen la base parametrizable del modelo.

En la segunda fase, se formaliza un modelo declarativo sobre el grafo acíclico dirigido que describe el entrenamiento supervisado, donde cada nodo representa una operación diferenciable y las aristas codifican dependencias topológicas. El modelo se formula como un sistema de Programación Lineal Entera con variables binarias que definen la asignación de operaciones a CPU o GPU, la retención o recomputación de activaciones y el uso opcional de técnicas asíncronas como streaming interdispositivo, rematerialización o prefetching. Las restricciones incluyen límites de memoria por dispositivo, preservación del orden de

ejecución, penalizaciones energéticas, latencias de transferencia y condiciones necesarias para el aprendizaje supervisado basado en gradientes.

La tercera fase consiste en parametrizar el modelo con las métricas reales obtenidas en la caracterización inicial, asegurando coherencia semántica entre observación y simulación, y minimizando divergencias entre ambos entornos. Se incluyen métricas de huella de memoria por capa, latencia de ejecución, consumo energético y limitaciones específicas. Este conjunto se complementa con un análisis de sensibilidad que varía sistemáticamente parámetros clave, penalizaciones por recomputación, latencia de transferencia y umbrales de memoria, para evaluar la estabilidad estructural de las soluciones y su capacidad de generalización.

Con el modelo parametrizado, en la cuarta fase se resuelven instancias mediante solvers ILP, derivando planes de ejecución que integran asignaciones CPU-GPU y políticas de gestión de memoria. A partir de cada solución, se implementa un simulador que reconstruye la secuencia de operaciones, verificando compatibilidad topológica, factibilidad técnica y cumplimiento de restricciones. El simulador estima el comportamiento agregado en términos de uso de recursos, penalizaciones temporales, solapamiento cómputo-comunicación y eficiencia de asignación. Estos resultados se validan de forma cruzada con ejecuciones físicas preliminares, ajustando discrepancias y comparando las estrategias híbridas con esquemas monolíticos centrados en GPU.

Finalmente, la quinta fase aborda la validación experimental en plataforma física, implementando las soluciones óptimas en PyTorch con asignación explícita de capas a CPU o GPU. Se miden uso activo de memoria, penalizaciones temporales, consumo energético estimado mediante telemetría interna y precisión final del modelo. Esta validación confirma tanto el rendimiento técnico como la coherencia metodológica del modelo, contrastando los resultados con esquemas tradicionales y verificando la solidez de sus supuestos en entornos heterogéneos reales.

En conjunto, este flujo metodológico integral y reproducible asegura que las decisiones de asignación optimizadas mantengan su viabilidad tanto en simulación controlada como en ejecución práctica, demostrando eficiencia operativa y consistencia de resultados bajo condiciones reales de cómputo heterogéneo.

1.8. 1.8 Contribucion cientifica

La contribución científica de esta investigación radica en la formalización, validación empírica y aplicación de un modelo declarativo de optimización a nivel de capa para el

entrenamiento de redes neuronales profundas en entornos heterogéneos CPU-GPU. Este modelo reinterpreta el entrenamiento supervisado como un problema estructurado de asignación, en el que las operaciones computacionales se redistribuyen de manera consistente, verificable y eficiente entre CPU y GPU, respetando las restricciones arquitectónicas observadas en plataformas reales.

A diferencia de los esquemas tradicionales centrados en la GPU, el enfoque propuesto busca reducir la presión de memoria activa sobre el acelerador mediante decisiones explícitas de rematerialización, desplazamiento computacional y sincronización entre dispositivos, preservando la exactitud del modelo y la viabilidad operativa. El sistema formal, basado en Programación Lineal Entera sobre grafos acíclicos de cómputo, integra métricas reales de ejecución tanto en la función objetivo como en las restricciones, garantizando que las soluciones generadas reflejen un comportamiento medible bajo condiciones reproducibles.

Metodológicamente, la investigación se apoya en la secuencia estructurada en la sección de metodología. Este proceso abarca la observación empírica, el modelado computacional, la resolución algorítmica, la simulación híbrida y la validación experimental. El flujo de trabajo está diseñado para ser replicable en distintos contextos arquitectónicos, diseños reales de redes neuronales y marcos de entrenamiento, y es extensible a sistemas multi-GPU, esquemas de memoria distribuida y arquitecturas heterogéneas emergentes con soporte para ejecución asíncrona en CPU.

Asimismo, el estudio introduce la noción operativa de sensibilidad arquitectónica, que evalúa la estabilidad de las soluciones óptimas frente a variaciones controladas en parámetros críticos, como latencia, penalizaciones energéticas y capacidad de retención de memoria. Este análisis no solo facilita la selección de configuraciones optimizadas, sino que también aporta información valiosa sobre el espacio de diseño técnico de los sistemas heterogéneos.

En consecuencia, la tesis presenta un marco computacional formalmente fundamentado y validado empíricamente para el entrenamiento de redes neuronales profundas con conciencia de recursos. Este marco permite razonar de forma sistemática sobre los compromisos entre memoria y rendimiento, apoya el desarrollo de estrategias de ejecución energéticamente eficientes y contribuye al rediseño de flujos de entrenamiento en entornos con saturación de GPU.

La originalidad de esta propuesta radica en integrar un modelo declarativo formal con métricas empíricas de ejecución para guiar la asignación óptima de operaciones en entornos heterogéneos. Su impacto potencial se extiende más allá del caso de estudio, ofreciendo a la comunidad de HPC y Deep Learning una herramienta metodológica replicable, adaptable

a nuevas arquitecturas y capaz de influir en el diseño futuro de sistemas y algoritmos de entrenamiento distribuidos.

1.9. 1.9 Organizacion del documento

La organización de la monografía responde al orden epistemológico del problema y no a una división meramente expositiva. El Capítulo 2 desarrolla los fundamentos teóricos que permiten comprender el entrenamiento profundo como un problema de ejecución sobre grafo, de jerarquía de memoria y de decisión bajo restricciones. El Capítulo 3 presenta la arquitectura metodológica del sistema como una cadena explícita de transformación de evidencia. El Capítulo 4 estudia el profiling empírico y la construcción de coeficientes observados. El Capítulo 5 formaliza la optimización ILP y sus mecanismos de robustificación, sensibilidad y comparación. El Capítulo 6 cierra el bucle entre planificación y ejecución mediante simulación y runtime híbrido. Finalmente, el Capítulo 7 sintetiza aportes, límites y proyección investigativa.

Esta secuencia es deliberada. Primero se fundamenta conceptualmente el problema; después se explica cómo se produce evidencia; luego se formula la decisión; y solo entonces se discute su ejecución real. En una tesis doctoral, esta progresión evita dos defectos comunes: teorizar sin anclaje experimental o describir resultados sin explicar con suficiente profundidad el marco conceptual que los hace significativos.

Capítulo 2

Marco Teorico y Fundamentacion del Problema

2.1. 2.1 Entrenamiento de redes profundas y grafos computacionales

El entrenamiento de una red profunda puede entenderse como la ejecución iterada de un grafo computacional dirigido donde cada nodo representa una operación tensorial y cada arista representa una dependencia de datos o de gradientes. Esta lectura es central para la tesis porque desplaza el análisis desde el rendimiento agregado del modelo hacia la estructura causal de su ejecución. En otras palabras, el problema no consiste únicamente en determinar cuánto cuesta entrenar un modelo completo, sino en comprender cómo se distribuye ese costo a través de una topología de dependencias cuyos nodos y aristas condicionan la factibilidad de toda política de colocación.

Formalmente, si x_0 es la entrada y f_ℓ la transformación asociada a la capa ℓ , la propagación hacia adelante puede representarse como

$$x_\ell = f_\ell(x_{pa(\ell)}; \theta_\ell),$$

donde $pa(\ell)$ denota el conjunto de predecesores de la capa ℓ en el grafo y θ_ℓ los parámetros locales. La retropropagación, a su vez, propaga gradientes a través de la regla de la cadena, lo cual obliga a preservar o reconstruir activaciones intermedias de manera coherente con la estructura del mismo grafo. Esta dependencia estructural es precisamente la razón por la cual la asignación de cómputo no puede decidirse de forma puramente nodal sin considerar el patrón de cortes inducidos entre operaciones relacionadas.

La representación por grafo resulta especialmente relevante en arquitecturas con conexiones residuales, bloques multi-rama o patrones no secuenciales, donde la linealización ingenua por orden de módulos puede ocultar la verdadera geometría del flujo de datos. Desde la perspectiva de esta tesis, ello implica que la fidelidad estructural del grafo no es un lujo descriptivo, sino una condición de validez del modelo de partición. Solo un grafo que preserve dependencias relevantes permite atribuir correctamente costos de comunicación y restricciones de memoria a la política de colocación resultante.

La importancia teórica de este punto es mayor de lo que podría parecer a primera vista. En muchos trabajos de ingeniería aplicada, el entrenamiento se resume en curvas agregadas de throughput o en tiempos por batch, con lo cual la estructura causal del cálculo desaparece bajo una media global. Sin embargo, una tesis doctoral orientada a la optimización de asignación necesita un nivel de resolución distinto: debe aislar las unidades efectivas de decisión, explicar cómo esas unidades interactúan y demostrar por qué una política válida en un subgrafo puede dejar de serlo cuando se consideran sus dependencias entrantes y salientes. El grafo computacional no es, por tanto, solo una representación conveniente, sino el soporte ontológico del problema de partición.

Desde esta perspectiva, la distinción entre forward y backward adquiere un estatuto central. El forward transforma entradas y construye activaciones; el backward reutiliza esa información para propagar gradientes y actualizar parámetros. Ambos recorridos comparten estructura, pero no necesariamente comparten costo, sensibilidad a memoria o conveniencia de asignación. Esta asimetría ayuda a justificar el alcance fuerte adoptado por la tesis: si la dinámica material de ambas fases no es idéntica, entonces una política de partición científicamente ambiciosa no debería imponer de antemano una asignación idéntica para las dos.

2.2. 2.2 Arquitecturas heterogeneas CPU-GPU y jerarquias de memoria

Una plataforma heterogénea CPU-GPU combina dos regímenes materiales de ejecución. La GPU concentra alto paralelismo y gran densidad de cómputo para operaciones tensoriales masivas, pero trabaja sobre una memoria de dispositivo limitada y de elevada sensibilidad a la saturación. La CPU ofrece menor throughput masivo para ciertos patrones de entrenamiento, pero dispone de mayor flexibilidad de control, un espacio de memoria principal sustancialmente más amplio y mecanismos vectoriales que pueden resultar útiles en determinadas porciones del grafo o como soporte de almacenamiento temporal.

Esta dualidad convierte la jerarquía de memoria en un elemento teórico central del problema. Si se denota por M_{param} la memoria de parámetros, por M_{grad} la memoria de gradientes, por M_{opt} la memoria de estados del optimizador y por M_{act} la memoria de activaciones, el consumo total de memoria relevante durante el entrenamiento puede aproximarse como

$$M_{total} \approx M_{param} + M_{grad} + M_{opt} + M_{act}.$$

En arquitecturas profundas, el término dominante para la factibilidad operativa suele ser M_{act} , especialmente durante la fase de retropropagación, cuando la regla de la cadena exige disponibilidad de representaciones intermedias. De ahí que la tesis entienda la memoria no como un mero criterio de post-filtrado, sino como una restricción constitutiva de la política de partición.

Bajo esta lectura, una política íntegramente orientada a GPU puede ser nominalmente atractiva en tiempo de cómputo y, sin embargo, inviable por desbordamiento de VRAM. Del mismo modo, una política excesivamente desplazada a CPU puede reducir presión de memoria y aumentar de forma inaceptable la latencia global. La tesis se sitúa precisamente en el espacio intermedio donde la heterogeneidad se vuelve una variable de decisión y donde la asignación óptima depende de negociar explícitamente computación, memoria y comunicación.

Esta negociación no es puramente cuantitativa; es también arquitectónica. La GPU y la CPU no son dos dispositivos intercambiables con distinta velocidad, sino subsistemas con semánticas operativas diferentes. La primera favorece kernels densos, paralelismo masivo y acceso local altamente estructurado; la segunda ofrece un espacio de control más versátil, mayor memoria direccionable y mejor tolerancia para ciertas operaciones de soporte o para estados que no requieren residencia permanente en el acelerador. Entender esta diferencia es esencial para evitar una visión reduccionista en la que la CPU aparezca solo como un refugio de emergencia cuando la VRAM se agota. En la lógica de esta tesis, la CPU debe interpretarse como un actor computacional de la política, no solo como almacenamiento pasivo.

Otro aspecto teórico clave es que la memoria no se experimenta como una magnitud uniforme. La memoria de parámetros, la de gradientes, la de estados del optimizador y la de activaciones tienen dinámicas temporales diferentes. Los parámetros suelen persistir a lo largo de toda la corrida; los gradientes aparecen y se consumen durante el ciclo de actualización; los estados del optimizador dependen del algoritmo utilizado; y las activaciones obedecen a una lógica de disponibilidad transitoria gobernada por la topología

del grafo. La consecuencia es inmediata: una política seria de optimización no puede tratar la memoria como un escalar indiferenciado, sino como una superposición de componentes con significado operativo distinto.

Si además se considera la escala de modelos contemporáneos, esta distinción se vuelve aún más importante. Vision Transformers, arquitecturas convolucionales tridimensionales y modelos multietapa presionan la jerarquía de memoria no solo por el número de parámetros, sino por el tamaño y la vida útil de las representaciones intermedias. Por eso, el problema de la tesis no se reduce a ejecutar “más rápido”, sino a reorganizar qué debe residir, dónde y durante cuánto tiempo para mantener la viabilidad del entrenamiento.

2.3. 2.3 Transferencia de datos, solapamiento y comunicación

En un sistema heterogéneo, toda asignación entre CPU y GPU induce la posibilidad de transferencias inter-dispositivo. Estas transferencias no son un efecto secundario menor, sino un costo estructural capaz de anular los beneficios locales de ejecutar ciertos nodos en el acelerador. Desde una perspectiva teórica, ello implica que el problema de colocación no puede formularse solo sobre costos nodales: debe incorporar también el costo de las aristas que se cortan entre dispositivos.

La tesis adopta para este fin una parametrización alpha-beta de primer orden, donde el tiempo de transferencia de un tensor de tamaño S puede modelarse como

$$t_{dir}(S) = \alpha_{dir} + \frac{S}{\beta_{dir}},$$

donde α_{dir} representa la latencia base de la dirección de transferencia y β_{dir} el ancho de banda efectivo. Esta expresión no agota toda la complejidad del subsistema de comunicación, pero ofrece una base interpretable y calibrable para integrar la transferencia en la función objetivo.

En presencia de streams o mecanismos de asincronía, parte de este costo puede solaparse con cómputo. Si $\sigma \in [0, 1]$ representa la proporción de solapamiento observada, el costo efectivo puede atenuarse mediante un factor de overlap $f_{ov}(\sigma)$, de manera que

$$t_{dir}^{eff}(S) = t_{dir}(S) f_{ov}(\sigma).$$

La importancia de este modelo teórico no radica solo en cuantificar comunicación, sino en introducir una intuición central para la tesis: una partición útil no se mide únicamente

Cuadro 2.1: Dimensiones teóricas que estructuran el problema de partición heterogénea.

Dimension	Significado dentro del problema doctoral
Grafo de entrenamiento	Representa la estructura causal de operaciones, dependencias y puntos potenciales de corte entre dispositivos.
Jerarquia de memoria	Introduce presupuestos fisicos que convierten la factibilidad en una condicion constitutiva de la politica de asignacion.
Transferencia inter-dispositivo	Expresa el costo comunicativo asociado a fragmentar el grafo entre CPU y GPU, evitando soluciones localmente optimas pero globalmente degradadas.
Persistencia de activaciones	Modela la disponibilidad temporal de informacion necesaria para backward y abre el espacio de compromiso entre memoria y recomputacion.
Optimizacion declarativa	Proporciona un lenguaje formal para traducir estos compromisos en decisiones binarias ejecutables y auditables.

por cuánto cálculo desplaza a GPU o CPU, sino por el equilibrio que establece entre computación local y costo de frontera entre ambos dispositivos.

Esta observación tiene una implicación metodológica decisiva. Una capa que en aislamiento sea claramente más rápida en GPU puede dejar de ser la mejor candidata para GPU cuando la política completa induce múltiples cortes de arista y, con ellos, una secuencia de transferencias costosas o mal solapadas. En consecuencia, la tesis rechaza la intuición miope según la cual basta asignar al acelerador las capas “más pesadas”. El verdadero problema es global y depende de la geometría del corte inducido sobre el grafo, no solo del rendimiento local de cada operador.

Conviene subrayar además que el solapamiento no elimina la comunicación; solo la recontextualiza. El hecho de que una transferencia pueda ocultarse parcialmente tras cómputo concurrente no la vuelve gratuita, y menos aún epistemológicamente irrelevante. Sigue siendo necesario modelar cuándo el solapamiento es factible, qué fracción del costo puede enmascarse y bajo qué supuestos de scheduler o de runtime ese beneficio resulta creíble. De ahí que la tesis trate *streaming* y *prefetching* como extensiones formales del problema, no como optimizaciones ad hoc agregadas al final del pipeline.

2.4. 2.4 Precision numerica, estabilidad y costo material del entrenamiento

Las politicas de precision numerica modifican de manera simultanea el perfil de memoria, la intensidad computacional y, potencialmente, la estabilidad del entrenamiento.

En terminos materiales, pasar de `fp32` a formatos de menor precision puede reducir el tamano de ciertos tensores y aumentar el throughput efectivo del hardware. En terminos metodologicos, sin embargo, la precision no puede tratarse como una constante abstracta ni como un simple interruptor de aceleracion: debe registrarse como parte de la condicion experimental porque altera tanto la semantica de los coeficientes observados como la comparabilidad entre corridas.

Si se denota por $b(p)$ el numero de bytes asociado al formato numerico p , una aproximacion simple al efecto de precision sobre una componente de memoria es

$$M^{(p)} \approx M^{(fp32)} \frac{b(p)}{b(fp32)}.$$

La expresion es deliberadamente simplificada, pero basta para mostrar que el problema de memoria no esta separado del regimen numerico de ejecucion. En plataformas reales, ademas, la precision efectiva depende de soporte ISA, backend y politica de autocast, de manera que la precision “solicitada” puede no coincidir con la precision realmente ejecutada. Por ello, el manuscrito considera la precision como una condicion de validez del profiling y no solo como un hiperparametro de conveniencia.

La tesis incorpora esta dimension en su marco teorico por una razon adicional. Una politica de particion que parezca favorable bajo un regimen numerico puede dejar de serlo bajo otro si cambian la huella de memoria, el tiempo por operador o la estabilidad de la actualizacion. En otras palabras, la precision forma parte del espacio de restricciones y no debe quedar fuera de la interpretacion de resultados. Este punto refuerza la idea de que el entrenamiento heterogeneo es un problema de sistemas integrados, donde decision de dispositivo, formato numerico y gestion de memoria coevolucionan.

2.5. 2.5 Estrategias de gestion de memoria

La gestión de memoria en entrenamiento profundo puede entenderse como un problema de administración de disponibilidad temporal de información. Las activaciones intermedias deben existir cuando el backward las requiere, pero no necesariamente deben permanecer siempre residentes en el mismo dispositivo ni conservarse mediante una única estrategia. A partir de esta observación emergen distintas políticas: retención completa, checkpointing, rematerialización, precisión reducida, offloading y prefetching.

Cada una de estas políticas representa un intercambio distinto entre memoria y tiempo. Si M_{save} denota memoria ahorrada por una estrategia de rematerialización y T_{recomp} el

tiempo extra de recomputación inducido, el problema deja de ser maximizar una sola métrica y pasa a negociar un compromiso entre recursos:

$$\Delta Z \sim \lambda_M M_{save} - \lambda_T T_{recomp},$$

donde λ_M y λ_T codifican la importancia relativa de memoria y tiempo en el régimen experimental considerado. La tesis adopta esta lógica de compensación explícita y la desplaza desde el nivel intuitivo hacia la formulación matemática y su validación experimental.

La consecuencia teórica más importante es que la memoria deja de ser tratada como un parámetro fijo del entorno y se convierte en una variable parcialmente gobernable por la política de ejecución. Esto justifica que la tesis incorpore persistencia de activaciones, recomputación y soporte de asincronía como parte del alcance fuerte del problema, en lugar de dejarlos como consideraciones externas al modelo principal.

En este punto resulta importante distinguir entre técnicas que solo desplazan el umbral de saturación y técnicas que cambian realmente la estructura del problema. La precisión mixta, por ejemplo, puede reducir el volumen de ciertos tensores; el *checkpointing* disminuye la memoria a costa de recomputación; y el *offloading* desplaza estado hacia otra memoria disponible. Pero ninguna de estas estrategias, por si sola, resuelve la pregunta normativa de *que* debe ejecutarse en cada dispositivo y *cundo* conviene hacerlo. Esa es precisamente la razón por la que el marco teórico de esta tesis necesita avanzar desde técnicas aisladas de alivio hacia una política combinatoria de asignación.

La rematerialización merece una atención especial porque transforma una intuición ingenieril en una variable teorizable. Conservar una activación o recomputarla deja de ser una decisión artesanal del programador y pasa a ser un compromiso formalizable entre memoria disponible y tiempo adicional. Esta es una de las transiciones conceptuales más importantes del manuscrito: el paso desde la optimización de bajo nivel como “truco” de implementación hacia su incorporación como decisión legítima dentro de un espacio declarativo de solución.

2.6. 2.6 Paradigmas contemporáneos de paralelización y sus límites

La literatura de entrenamiento a gran escala ha desarrollado varios paradigmas de paralelización: paralelismo de datos, paralelismo de modelo, particionamiento por etapas y estrategias híbridas entre ellos. Todos ellos responden a problemas reales y han demostrado

Cuadro 2.2: Paradigmas de paralelizacion y razon de insuficiencia frente al problema doctoral.

Paradigma	Limite respecto al problema de esta tesis
Paralelismo de datos	Replica el modelo pero no reduce de forma estructural la presion de memoria de cada replica individual sobre una unica GPU.
Paralelismo de modelo	Fragmenta el modelo entre dispositivos, pero no necesariamente incorpora una formulacion empirica y declarativa especifica para cooperacion CPU-GPU local.
Pipeline por etapas	Optimiza el flujo temporal de microbatches, pero no agota el problema de asignacion fina de nodos, activaciones y cortes dentro de una sola plataforma heterogenea.
Tecnicas locales de memoria	Alivian sintomas de saturacion, pero no sustituyen una politica global de decision sobre dispositivo, persistencia y transferencia.

utilidad en determinados regimenes de hardware y de tamano de modelo. Sin embargo, su existencia no elimina el vacio que organiza esta tesis, porque la mayor parte de esos enfoques se centra en distribuir carga entre multiples aceleradores o en replicar estado entre nodos, no en reorganizar de forma fina la cooperacion CPU-GPU dentro de una misma plataforma heterogenea con criterios explicitos de memoria, transferencia y validez operacional [3, 6, 10, 11].

El paralelismo de datos replica el modelo en multiples dispositivos y distribuye batches o microbatches entre ellos. Su fortaleza radica en la simplicidad conceptual y en su buena escalabilidad cuando la sincronizacion de gradientes es razonable. Pero no resuelve de manera directa la saturacion de memoria de una unica GPU sobre la que sigue residiendo cada replica. El paralelismo de modelo, por su parte, divide partes de la red entre dispositivos distintos, pero suele exigir acoplamientos de comunicacion complejos y no incorpora necesariamente una formulacion declarativa que optimice la asignacion bajo coeficientes empiricos capa a capa [3, 6, 10].

La particion por etapas y los enfoques tipo pipeline abordan el problema desde la perspectiva del flujo temporal de microbatches. Aunque ofrecen mecanismos de solapamiento utiles, su foco principal no coincide con el de esta tesis: aqui el interes no es solamente throughput en una topologia multiacelerador, sino la redistribucion metodologicamente justificada de computo y estado entre CPU y GPU dentro de un grafo de entrenamiento con restricciones de memoria y con necesidad de trazabilidad completa.

La consecuencia teorica de este contraste es clara. El problema doctoral no consiste en elegir entre paradigmas establecidos, sino en formular una capa adicional de decision que los complemente desde una logica de asignacion heterogenea local, observacionalmente

calibrada y operacionalmente verificable. Ese es el nicho conceptual en el que se inserta la contribucion del manuscrito.

Conviene insistir en que este posicionamiento no equivale a negar la utilidad de las estrategias existentes. Al contrario, la tesis se apoya en ellas como marco de contraste para identificar con mayor precision su propio vacio. El paralelismo de datos muestra que la replicacion de modelo resuelve ciertas necesidades de throughput; el paralelismo de modelo evidencia que la distribucion de estado puede ser necesaria para arquitecturas grandes; y las tecnicas de optimizacion de memoria revelan que el problema de residencia de activaciones es genuino y no anecdotico. Lo que ninguna de estas familias resuelve de manera simultanea es la articulacion entre medicion empirica fina, decision declarativa interpretable y cierre experimental CPU-GPU en una sola plataforma.

Desde esa perspectiva, la contribucion teorica del manuscrito puede leerse como una tesis de integracion. No propone simplemente otro truco de ahorro de memoria ni otra forma de distribuir lotes, sino una politica de asignacion que se deja estudiar como objeto cientifico completo: tiene insumos observacionales, una formulacion matematica, un espacio de sensibilidad y una semantica operacional verificable. Este es el punto en que el marco teorico deja de ser una lista de antecedentes tecnicos y pasa a cumplir su funcion doctoral: justificar por que la pregunta de investigacion merece un tratamiento propio y no puede absorberse sin residuo en paradigmas ya consolidados.

2.7. 2.7 Optimizacion declarativa: ILP en sistemas computacionales

La programación lineal entera es especialmente adecuada cuando el problema involucra decisiones discretas, restricciones físicas explícitas y necesidad de interpretabilidad causal. En el caso que ocupa esta tesis, la asignación de una capa a CPU o GPU no es una variable continua ni una preferencia blanda: es una decisión ejecutable y binaria. Por ello, el ILP ofrece un lenguaje natural para expresar la política de colocación bajo restricciones y con significado operacional claro.

Si x_v denota la asignación del nodo v e y_{uv} el corte inducido en la arista (u, v) , el núcleo del problema consiste en minimizar una función objetivo lineal de la forma

$$\text{mín } Z = \sum_{v \in V} [x_v C_{gpu}(v) + (1 - x_v) C_{cpu}(v)] + \sum_{(u,v) \in E} y_{uv} C_{cut}(u, v),$$

sujeta a restricciones de memoria y a un sistema de linealización que fuerce la equivalen-

Cuadro 2.3: Comparacion conceptual entre familias de soluciones existentes y la orientacion de esta tesis.

Familia de soluciones	Fortaleza principal	Aspecto que la tesis toma como vacio
Paralelismo de datos	Escalado simple de throughput sobre multiples aceleradores	No convierte la cooperacion CPU-GPU local en problema explicito de decision capa a capa
Paralelismo de modelo o pipeline	Reparte estado o flujo de microbatches entre dispositivos	No ofrece por si mismo un marco empirico-declarativo centrado en memoria, transferencia y trazabilidad local
Tecnicas de ahorro de memoria	Reducen saturacion mediante checkpointing, precision o offloading	Suelen operar como mecanismos parciales, no como politica global de asignacion con validacion formal y runtime
Heurísticas locales de placement	Son faciles de aplicar y de bajo costo computacional	Carecen de interpretacion global robusta cuando existen cortes costosos y restricciones acopladas

cia lógica entre y_{uv} y la diferencia binaria $|x_u - x_v|$. Esta estructura es importante porque preserva simultáneamente tractabilidad matemática e interpretación física. El modelo no solo entrega una solución; entrega una explicación explícita sobre cómo se compone el costo total entre cómputo local y fragmentación comunicativa.

La tesis adopta además una robustificación estadística de coeficientes a través de la transformación

$$\hat{m} = \mu_m + k_\sigma \sigma_m,$$

donde μ_m y σ_m son media y dispersión de la métrica observada, y k_σ controla el grado de conservadurismo operacional. Este paso es teóricamente importante porque introduce incertidumbre observada dentro del problema declarativo, evitando que la optimización se apoye exclusivamente en medias puntuales potencialmente frágiles.

La eleccion del ILP no debe interpretarse como una preferencia meramente tecnica por un solver conocido, sino como una toma de posicion epistemologica. Una tesis doctoral necesita un modelo cuya semantica sea interpretable y cuyas restricciones puedan leerse en clave fisica. El ILP cumple esa exigencia porque obliga a declarar de forma explicita que decisiones son binarias, que recursos son limitados y que compromisos se estan aceptando

al pasar de una solución factible a otra. La solución no emerge de una caja negra estadística difícil de auditar, sino de una estructura matemática en la que cada término de la función objetivo y cada restricción mantienen significado experimental.

Además, el marco declarativo permite extender el problema sin descomponer su coherencia interna. Asignación diferenciada para *forward* y *backward*, persistencia de activaciones, penalizaciones de transferencia, presupuestos de memoria y soporte para asincronía pueden integrarse como familias adicionales de variables y restricciones. Esta extensibilidad es crucial para la tesis, porque evita que el modelo base se convierta en un callejón sin salida incapaz de absorber la versión fuerte del alcance doctoral.

Otra ventaja teórica del enfoque declarativo es su capacidad de hacer visible lo que otras aproximaciones dejan implícito. Cuando una política se expresa como conjunto de restricciones y términos de costo, el lector puede interrogar directamente el modelo: que se privilegia, que se penaliza, que se acepta como sacrificio razonable y que se considera inviable. Esta transparencia es especialmente valiosa en una tesis doctoral porque desplaza la discusión desde el simple hecho de obtener una solución hacia la pregunta más exigente de por qué esa solución debe considerarse aceptable dentro del problema definido.

2.8. 2.8 Síntesis del marco teórico y delimitación del vacío metodológico

Los fundamentos presentados en este capítulo permiten delimitar con precisión el problema que la tesis aborda. El entrenamiento profundo puede interpretarse como ejecución sobre grafo; la jerarquía CPU-GPU introduce una heterogeneidad material de cómputo y memoria; las transferencias entre dispositivos convierten la comunicación en una dimensión de costo tan real como la latencia local; las políticas de precisión alteran tanto el consumo de recursos como la validez comparativa de las mediciones; y las estrategias de persistencia de activaciones muestran que la memoria es parcialmente gobernable por la política de ejecución. Bajo esta lectura, la partición heterogénea deja de ser una idea heurística y pasa a ser un problema formal de decisión sobre recursos.

El vacío que organiza la investigación no se limita a la existencia abstracta de ese problema, sino a la falta de un cierre metodológico completo entre fundamento teórico, observación empírica, modelado declarativo y validación operacional. Los paradigmas contemporáneos de paralelización resuelven partes del problema; las técnicas de memoria alivian síntomas concretos; y los enfoques de entrenamiento distribuido ofrecen escalabilidad en otros regímenes. Lo que permanece insuficientemente resuelto es la construcción de una

politica heterogenea CPU-GPU local, sustentada en coeficientes observados, formulada como optimizacion interpretable y verificada despues en simulacion y runtime.

La tesis se propone cubrir precisamente ese vacio: no solo formular una politica, sino construir una cadena consistente que explique por que esa politica es fisicamente significativa, matematicamente resoluble y experimentalmente contrastable. En este sentido, el marco teorico no opera como un preambulo decorativo, sino como el lugar donde se justifica que el problema merece una solucion doctoral propia y que dicha solucion no puede reducirse ni a benchmarking agregativo ni a una simple descripcion de software.

Con ello, el capitulo establece una conclusion fuerte para el resto del manuscrito. Si el problema ha sido correctamente delimitado, entonces los capitulos siguientes no pueden limitarse a mostrar que el software funciona. Deben demostrar que la arquitectura metodologica, el profiling, la optimizacion y el runtime responden precisamente a las tensiones conceptuales aqui identificadas. El marco teorico, por tanto, no antecede pasivamente al resto de la tesis: le impone un criterio de coherencia.

2.9. 2.9 Estado del arte comparativo en clave problematica

Aunque la presente version del manuscrito no incorpora aun una revision bibliografica exhaustiva con aparato completo de referencias, si permite formular una comparacion doctrinal entre familias de trabajo existentes y la posicion especifica de la tesis. De manera general, pueden distinguirse cuatro corrientes cercanas al problema. La primera agrupa enfoques orientados al escalado del entrenamiento mediante paralelismo de datos o de modelo; la segunda, tecnicas de optimizacion de memoria dentro del acelerador; la tercera, heurísticas de placement o scheduling sobre grafos de computo; y la cuarta, aproximaciones predictivas u offline que estiman costo sin cerrar el bucle en runtime. Cada una aporta intuiciones relevantes, pero ninguna agota por si sola el problema que aqui se formula.

Los enfoques de paralelismo distribuido han mostrado que el entrenamiento profundo puede reorganizarse sobre multiples dispositivos para aumentar throughput o acomodar modelos mayores. Sin embargo, su pregunta rectora no coincide con la de esta tesis. Suelen partir de un escenario multiacelerador y concentrarse en replicacion, fragmentacion de parametros o flujo de microbatches, mientras que el presente trabajo interroga la cooperacion local entre CPU y GPU dentro de una misma plataforma heterogenea y la convierte en decision explicita bajo restricciones observadas. La diferencia no es menor: cambia tanto la unidad de decision como el tipo de evidencia requerido para sostenerla.

Las técnicas de ahorro de memoria, por otra parte, constituyen un antecedente indispensable pero insuficiente. Checkpointing, rematerialización, precisión mixta y offloading muestran que la saturación de VRAM puede mitigarse manipulando la disponibilidad temporal de activaciones o el formato numérico de ciertos tensores [2, 1, 8, 9]. No obstante, en muchos casos estas técnicas aparecen como mecanismos locales acoplados a implementaciones específicas, sin un marco unificado que explicita cuando conviene aplicarlas, que costo global introducen y como se relacionan con una política más amplia de asignación inter-dispositivo. La tesis recoge esas intuiciones, pero las reubica dentro de una estructura de decisión más general.

Una tercera familia de trabajos se orienta al placement o scheduling sobre grafos, a veces con heurísticas y a veces con formulaciones de optimización. Su valor principal reside en reconocer que el grafo, y no solo el operador aislado, debe guiar la decisión. Sin embargo, muchas de estas propuestas carecen de dos elementos que para esta tesis son nucleares: primero, una parametrización empírica robusta derivada de profiling reproducible bajo precisión y hardware efectivamente ejecutados; segundo, una validación que no se detenga en el solver o en la simulación, sino que continúe hasta runtime con trazabilidad de plan [6, 11]. En ese punto se localiza una parte esencial del aporte doctoral del manuscrito.

Por último, existen aproximaciones predictivas que modelan costo de ejecución y permiten comparar estrategias de manera offline. Tales modelos son útiles para explorar espacios de decisión amplios sin consumir recursos físicos en cada intento. Pero si el bucle se cierra exclusivamente en la predicción, queda abierta la pregunta decisiva de toda tesis de sistemas: hasta qué punto la política optimizada conserva su significado cuando se implementa de verdad. La presente investigación se distancia de ese límite al insistir en que el valor científico de una política heterogénea no se agota en su elegancia matemática ni en su curva de simulación, sino en la continuidad verificable entre formulación, predicción y observación material [6, 11].

Leído en conjunto, este estado del arte comparativo en clave problemática permite situar con mayor nitidez la singularidad del trabajo. La tesis no compite simplemente con una técnica concreta de memoria o con un esquema concreto de paralelización. Su apuesta es más estructural: construir una política heterogénea CPU-GPU fundada en medición por capa, expresada como optimización interpretable y verificada a través de una cadena completa de evidencia. Ese desplazamiento del problema, desde la mejora local hacia el cierre metodológico integral, es la razón por la cual el manuscrito reclama un espacio propio dentro del panorama de soluciones existentes.

Nota de borrador: La bibliografía incluida en esta versión cumple una función de anclaje

comparativo y debe ampliarse en la version de deposito con una cobertura mas extensa de literatura especializada.

Capítulo 3

Arquitectura Metodologica del Sistema

3.1. 3.1 Vision general del pipeline experimental

La arquitectura metodológica del sistema no debe confundirse con un inventario de módulos de software. Su función en la tesis es mostrar cómo una pregunta científica sobre asignación heterogénea se traduce en un procedimiento experimental capaz de producir evidencia acumulativa. El sistema se organiza, por tanto, como una secuencia de operadores de transformación de información: observación del comportamiento del entrenamiento, agregación robusta de mediciones, formulación de la instancia de decisión, simulación de planes, ejecución física de políticas y consolidación de resultados comparativos.

Esta lógica puede expresarse como una cadena

$$(M, H, C) \xrightarrow{\mathcal{P}} \mathcal{D}_{prof} \xrightarrow{A} \hat{\mathcal{C}} \xrightarrow{\mathcal{O}} \Pi \xrightarrow{\mathcal{S}} \hat{Y} \xrightarrow{\mathcal{E}} Y,$$

donde M denota el modelo, H la plataforma hardware y C la configuración experimental; $\mathcal{D}_{prof}$ es el conjunto de artefactos de profiling; $\hat{\mathcal{C}}$ el conjunto de coeficientes robustificados; Π el espacio de planes obtenidos por optimización; $\hat{Y}$ la predicción del simulador; y Y la observación del runtime. El valor de esta formulación es metodológico: deja claro que el resultado final de la tesis no emerge de un único bloque, sino de una composición ordenada de etapas con significado experimental diferenciado.

Tal arquitectura responde a una exigencia epistemológica concreta. Ninguna conclusión relevante debe depender de un paso opaco o no reconstruible. Por ello, cada transformación publica artefactos persistentes que pueden auditarse de forma independiente. El pipeline

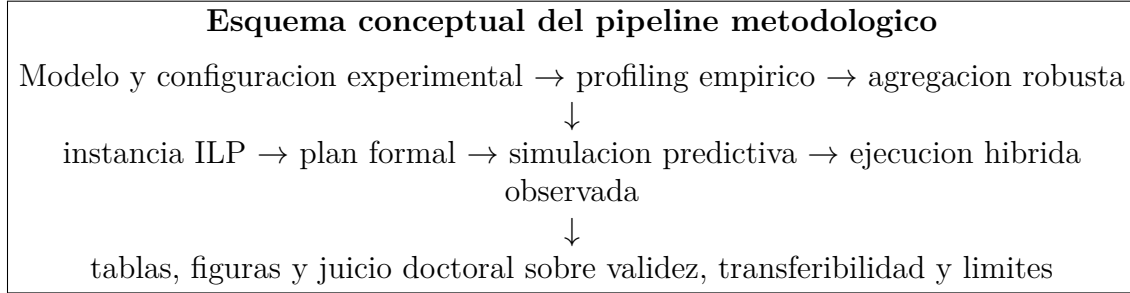


Figura 3.1: Figura conceptual del flujo de transformacion de evidencia en la tesis.

produce no solo un resultado final, sino una cadena intermedia de evidencia que permite localizar, justificar o cuestionar cada afirmación del manuscrito.

3.2. Estructura modular del software

La estructura modular del proyecto refleja una división de trabajo científica y no solo informática. El núcleo del sistema concentra capacidades de observación, modelado y ejecución; las capas de validación convierten esas capacidades en evidencia reproducible; y la orquestación de campañas asegura repetibilidad operacional. En esta tesis, esa separación es fundamental porque distingue tres planos que a menudo se confunden: implementar una capacidad, verificar su coherencia y usarla dentro de una campaña defendible.

Desde esta perspectiva, la importancia de la modularidad no radica únicamente en facilitar mantenimiento del código. Su función principal es preservar legibilidad metodológica. Un componente de profiling debe poder evolucionar sin reescribir la formulación ILP; un simulador debe poder contrastar planes sin alterar el runtime; y una ruta de validación debe poder ejercer como interfaz experimental estable aun cuando el núcleo técnico se refine internamente. Esta disciplina arquitectónica permite que la tesis argumente sobre subsistemas con fronteras conceptuales claras y no como si se tratara de una acumulación indiferenciada de scripts.

La monografía, en consecuencia, utiliza el software como soporte de una arquitectura metodológica más amplia. Los directorios y módulos importan porque materializan funciones científicas distintas: medir, transformar, decidir, predecir, ejecutar y reportar. El manuscrito no debe quedarse en su enumeración, sino interpretar qué papel cumple cada familia de componentes dentro de la cadena de construcción de evidencia.

Esta observacion merece subrayarse porque evita un deslizamiento frecuente en manuscritos tecnicos extensos: convertir la arquitectura en una taxonomia de ficheros. En una tesis doctoral, la estructura del software solo es relevante en la medida en que expresa

una division del trabajo científico. La pregunta correcta no es simplemente que modulo hace cada cosa, sino que responsabilidad epistemologica asume dentro del pipeline. Un componente de profiling responde por la validez observacional; uno de agregacion, por la transformacion inferencial del dato; uno de optimizacion, por la explicitud de la decision; y uno de runtime, por la correspondencia entre politica y ejecucion material.

3.3. 3.3 Contrato de artefactos y trazabilidad

El contrato de artefactos constituye la columna vertebral de la reproducibilidad del sistema. Un artefacto no es solo un archivo de salida: es una unidad documentada de evidencia que debe poder ser interpretada por humanos, consumida por etapas automáticas y situada causalmente dentro de una campaña experimental. Esta triple exigencia obliga a que cada producto intermedio conserve información suficiente sobre origen, condición de generación y función dentro del pipeline.

Si se denota por A_k el artefacto emitido por la etapa k , la trazabilidad exigida por la tesis puede resumirse como una relación de herencia de contexto

$$A_{k+1} = T_k(A_k, \Gamma_k),$$

donde T_k es la transformación aplicada en la etapa k y Γ_k representa el contexto experimental asociado: host, precisión efectiva, dataset, política de ejecución, procedencia del grafo y calibración de transferencia. La idea central es que ninguna transformación debe destruir la posibilidad de reconstruir de dónde proviene una decisión posterior.

En términos prácticos, esta filosofía se materializa en artefactos de profiling, agregación, solución ILP, simulación y ejecución híbrida. Pero el punto verdaderamente importante para la tesis es otro: gracias a este contrato, una mejora cuantitativa puede rastrearse hasta las mediciones y supuestos que la produjeron. En una investigación doctoral de sistemas, esa capacidad de reconstrucción causal es tan relevante como la mejora misma, porque distingue una conclusión reproducible de un resultado aislado difícil de auditar.

Desde aquí puede formularse una idea central para todo el manuscrito: la trazabilidad no es un anexo administrativo de la reproducibilidad, sino la forma concreta que adopta la inteligibilidad del sistema. Un artefacto trazable permite reconstruir no solo de donde viene un numero, sino bajo que compromisos experimentales fue posible producirlo. Esa reconstruccion es indispensable cuando la tesis discute sensibilidad, amenazas a la validez o diferencias entre simulacion y runtime, porque sin ella cualquier discrepancia podria atribuirse de manera arbitraria a etapas ocultas o decisiones no documentadas.

Cuadro 3.1: Correspondencia entre etapa metodologica y tipo de evidencia producida.

Etapa	Pregunta que responde	Evidencia principal
Profiling	Que costo observacional tiene cada unidad del grafo	Metricas crudas, metadatos y artefactos estructurales
Agregacion	Que coeficientes son metodologicamente admisibles	Estadisticos robustos, dispersion y banderas de calidad
Optimizacion	Que politica minimiza el costo bajo restricciones	Asignaciones, cortes y resumen de solucion
Simulacion	Que comportamiento se espera antes de ejecutar	Costos predichos, chequeos topologicos y presupuestos
Runtime hibrido	Como se comporta la politica en hardware real	Trazas observadas, memoria, energia y metrica final de calidad

3.4. 3.4 Definicion de plan y su representacion

El concepto de plan es el punto de unión entre el modelo declarativo y la ejecución física. Metodológicamente, un plan representa la hipótesis operacional concreta derivada de la optimización. No se reduce a un vector de asignaciones; condensa la política de ejecución que será sometida a predicción y observación. Por ello, en su forma más general puede expresarse como una tupla

$$\Pi = (X, Y, B, R, Q),$$

donde X representa asignaciones nodales, Y cortes de arista, B presupuestos observados de memoria, R reglas de retención o recomputación de activaciones y Q metadatos de procedencia y selección del plan. Esta estructura resume el hecho de que la solución de la tesis no es un número, sino una política ejecutable con semántica explícita.

La relevancia metodológica del plan radica en impedir el salto informal desde la solución matemática hacia una implementación ad hoc del ejecutor. Simulador y runtime deben consumir la misma semántica de plan para que el contraste entre predicción y observación tenga valor científico. Si el significado operativo cambiara entre ambos pasos, la tesis perdería continuidad causal y la comparación dejaría de ser interpretable.

En consecuencia, la representación de plan cumple también una función epistemológica: fija el lenguaje común entre optimización, simulación y ejecución. Gracias a ello, cada diferencia entre costo previsto y costo observado puede atribuirse a propiedades del sistema y no a una redefinición silenciosa de la política entre etapas.

La importancia de este punto aumenta cuando se considera la ambicion del cierre doctoral. Si la politica que resuelve el ILP, la politica que consume el simulador y la

politica que ejecuta el runtime no fueran estrictamente comparables, el bucle completo del manuscrito se fracturaria. La arquitectura metodologica del sistema evita precisamente ese problema al imponer una representacion intermedia suficientemente rica para ambos mundos: matematica para preservar semantica de optimizacion y operacional para sostener instrumentacion y contraste fisico.

3.5. 3.5 Criterios de validez y riesgos metodologicos iniciales

La arquitectura metodológica se apoya en compuertas de validez que delimitan qué evidencia puede considerarse doctoralmente admisible. Entre ellas destacan la exigencia de procedencia estructural suficiente del grafo, la necesidad de calibración de transferencia empíricamente medida, la preservación explícita de la precisión realmente ejecutada y la suficiencia estadística mínima de las campañas de profiling. Tales compuertas no son restricciones burocráticas, sino salvaguardas contra la inflación narrativa del resultado.

Los riesgos metodológicos iniciales se distribuyen en tres niveles. El primero es riesgo de medición: ruido, jitter, sensores imperfectos o degradación silenciosa de precisión pueden sesgar los coeficientes de entrada. El segundo es riesgo de modelado: una formulación elegante alimentada por insumos deficientes sigue siendo epistemológicamente débil. El tercero es riesgo de generalización prematura: extraer conclusiones doctorales amplias a partir de una base experimental local o insuficientemente replicada.

Puede resumirse esta lógica de admisibilidad mediante un predicado de validez experimental

$$\text{Val}(A) = \text{Qual}(A) \wedge \text{Trace}(A) \wedge \text{Struct}(A) \wedge \text{Exec}(A),$$

donde Qual evalúa suficiencia muestral y consistencia estadística, Trace la trazabilidad del contexto, Struct la validez estructural del grafo y Exec la correspondencia entre configuración declarada y ejecución efectiva. La tesis no necesita implementar este predicado como fórmula ejecutable para que su sentido sea científicamente útil: expresa la condición bajo la cual un artefacto deja de ser solo un producto técnico y pasa a ser una pieza aceptable de evidencia.

Nota de borrador: En la version final conviene insertar un diagrama arquitectonico del pipeline completo, con flechas de transformacion entre profiling, agregacion, optimizacion, simulacion, runtime y consolidacion de reporte.

Hasta que ese diagrama quede consolidado con estilo final, la figura conceptual ya incluida y las tablas de correspondencia del capítulo cumplen una función transitoria importante: fijan el principio de que la arquitectura debe leerse como una teoría de transformación de evidencia y no como un repertorio de implementaciones. Ese principio es el que permite que los capítulos posteriores se lean de manera acumulativa y no como piezas técnicas desconectadas.

Capítulo 4

Profiling Empirico para Particion Heterogenea CPU-GPU

4.1. 4.1 Motivacion empirica y preguntas de medicion

El profiling constituye la base empirica del resto del programa doctoral. Sin coeficientes de tiempo, energia, memoria y transferencia obtenidos de forma reproducible, cualquier formulacion de particion corre el riesgo de ser formalmente correcta pero operacionalmente fragil. Por ello, este capitulo no trata el profiling como una etapa auxiliar de benchmarking, sino como un mecanismo de inferencia experimental cuyo producto final son coeficientes robustos y auditables para optimizacion.

Las preguntas de medicion que guian el diseno son cinco. Primero, como medir costo por capa sin colapsar la heterogeneidad en promedios de modelo completo. Segundo, como separar en la medida de lo posible computo efectivo de overhead de despacho. Tercero, como representar transferencia inter-dispositivo de forma util para penalizar cortes de grafo. Cuarto, como robustificar las mediciones ante ruido y variabilidad inter-replica. Quinto, como preservar trazabilidad suficiente para que cada coeficiente pueda reconstruirse desde los artefactos fuente.

4.2. 4.2 Arquitectura de instrumentacion

La arquitectura de instrumentacion se organiza alrededor del subsistema de profiling del proyecto y delega la captura operativa en el entrenador instrumentado y en componentes de soporte para politicas de precision, utilidades de sistema y extraccion estructural del grafo. El pipeline sigue una secuencia clara: parseo y normalizacion de entorno, evaluacion

de politica de precision, carga de entradas preferentemente desde datasets reales, registro de hooks por modulo hoja, ejecucion de replicas y consolidacion de salidas tabulares y JSON. En terminos de implementacion, esta arquitectura se apoya en las capacidades de instrumentacion y ejecucion dinamica de PyTorch, pero las reordena hacia un contrato de evidencia mas estricto que el habitual en bibliotecas de entrenamiento generalista [7].

La instrumentacion por modulo hoja evita el doble conteo en estructuras anidadas y permite asociar a cada unidad de decision su costo observacional. Para cada capa se registran tiempos, energia, huellas de memoria y estimaciones de FLOPs; cuando existe soporte GPU, tambien se incorporan eventos y sincronizaciones necesarias para separar tiempo de pared observado y tiempo de kernel. Esta decision metodologica favorece una granularidad alineada con el problema de particion sin descender a un nivel de kernel primitivo que aumentaria ruido y complejidad semantica.

4.3. 4.3 Modelo de energia y carga computacional

El proyecto aproxima la energia mediante potencia media observada por ventana temporal, lo que no constituye una medicion fisica perfecta por subsistema pero si una base comparativa consistente entre corridas. Esta aproximacion se combina con una heuristica de backward por capa y con formulas analiticas de FLOPs para los operadores dominantes, produciendo asi un conjunto de indicadores de costo utiles tanto para optimizacion como para interpretacion.

La inclusion simultanea de energia y carga computacional persigue un objetivo dual. Por un lado, amplifica la interpretabilidad del costo nodal mas alla de la mera latencia. Por otro, ayuda a distinguir capas aparentemente lentas por razones distintas: ineficiencia de kernel, saturacion de memoria o intensidad aritmetica insuficiente respecto al hardware local. En una tesis aplicada, esta contextualizacion es esencial para evitar conclusiones causales demasiado pobres.

4.4. 4.4 Politica de precision y validez

Una fuente clasica de invalidez experimental en hardware heterogeneo es asumir que la precision solicitada coincide con la precision realmente ejecutada. En CPU modernas, ello depende de soporte ISA explicito; en GPU, de disponibilidad y comportamiento efectivo del backend. El pipeline resuelve este riesgo mediante sondeo de capacidades y metadatos de politica, registrando campos como `allowed`, `cpu_precision_executed`, `reason` y `status`

para cada corrida o para cada omision justificada.

Esta compuerta de validez tiene tres efectos metodologicos. Primero, evita mezclar bajo una misma etiqueta mediciones obtenidas en regimens numericos distintos. Segundo, deja una huella auditable cuando una configuracion no puede ejecutarse. Tercero, reduce el riesgo de que los capitulos posteriores alimenten el ILP con coeficientes derivados de una precision nominal pero no efectiva.

4.5. 4.5 Extraccion estructural del grafo

La extraccion estructural del grafo utiliza `torch.fx` como ruta principal y `torch.export` para ciertos modelos decoder-only que requieren una via alternativa. El objetivo no es producir un diagrama cosmetico del modelo, sino un conjunto de nodos y aristas con metadatos suficientes para enlazar nombres de capa, huellas de activacion y dependencias tensoriales. El campo `graph_trace_source` preserva la procedencia estructural de cada artefacto, lo cual permite distinguir evidencia metodologicamente admisible de rutas puramente diagnosticas. Esta exigencia de fidelidad estructural se alinea con la experiencia acumulada en trabajos de colocacion automatica y paralelizacion compilada, donde pequenas diferencias en la representacion del grafo alteran significativamente la calidad de la decision [5, 4, 11].

Esta seccion tiene relevancia doctoral porque el costo de transferencia no puede formularse con rigor si la topologia se reduce a una cadena lineal artificial. Las conexiones residuales, los saltos y los patrones no secuenciales son precisamente los lugares donde una particion ingenua suele fallar. Por ello, el proyecto endurece la admisibilidad del grafo y relega el fallback lineal a un uso excepcional, no apto para sostener conclusiones principales.

4.6. 4.6 Modelo de transferencia consciente de arista

El modelo de transferencia adoptado sigue una parametrizacion alpha-beta por direccion, distinguiendo host-a-dispositivo y dispositivo-a-host. Esta diferenciacion es necesaria porque las plataformas reales no suelen ofrecer simetria perfecta de ancho de banda y latencia, y una calibracion simetrica introduciria sesgo sistematico en el costo de corte. Sobre esta base se aplica una atenuacion conservadora por overlap, de modo que el costo efectivo no ignore la posibilidad de solapamiento entre computo y comunicacion, pero tampoco incurra en optimismo excesivo. Esta opcion metodologica dialoga con la literatura de pipeline y

paralelizacion automatizada, donde el solapamiento parcial entre computo y transferencia es una fuente recurrente de divergencia entre costo ideal y costo materializado [3, 6, 11].

El resultado final es un escalar de transferencia por arista consumible por el ILP. Su valor metodologico radica en traducir un fenomeno fisico del sistema en una magnitud optimizable, sin perder la posibilidad de auditar la procedencia de cada parametro. Este equilibrio entre realismo y tractabilidad es una de las piezas que distinguen al pipeline frente a un benchmarking tradicional por lotes completos.

4.7. 4.7 Agregacion estadistica robusta

La agregacion de replicas se realiza sobre claves experimentales completas para evitar mezclar condiciones heterogeneas. Para cada canal numerico se calculan media, desviacion estandar y cuantiles superiores, y los coeficientes entregados al ILP se robustifican mediante el esquema $\hat{m} = \mu_m + k_\sigma \sigma_m$. Esta transformacion no es un detalle numerico menor, sino una declaracion metodologica explicita sobre el nivel de riesgo operacional que el experimento esta dispuesto a tolerar.

La presencia de `n_runs`, coeficientes de variacion y banderas de calidad convierte la agregacion en una compuerta inferencial. El sistema no solo resume mediciones; informa cuan confiable es el resumen. Esta decision es especialmente importante en resultados smoke, donde la dispersion puede ser alta y donde la tesis no debe confundir evidencia exploratoria con cierre estadistico definitivo.

4.8. 4.8 Validez experimental y reproducibilidad

La validez interna del profiling se protege mediante replicas, control de semillas, politicas de preflight y exclusion explicita de corridas invalidas. La validez de construccion se fortalece al combinar tiempo, energia, memoria, transferencia e intensidad de computo, evitando que una unica metrica absorba por completo la semantica del problema. La validez externa, por su parte, se aborda mediante normalizacion por host y mediante la posibilidad de fusion multi-hardware de coeficientes, paso indispensable para cerrar la tesis con alcance transversal.

Como resultado del trabajo ya consolidado en `docs/CAPITULO_TESIS_PROFILING_ES.md`, este capitulo deja establecido que el profiling no es una fase preliminar prescindible. Es el fundamento empirico de la decision combinatoria posterior y, por tanto, una condicion necesaria para toda afirmacion sobre bondad de la particion.

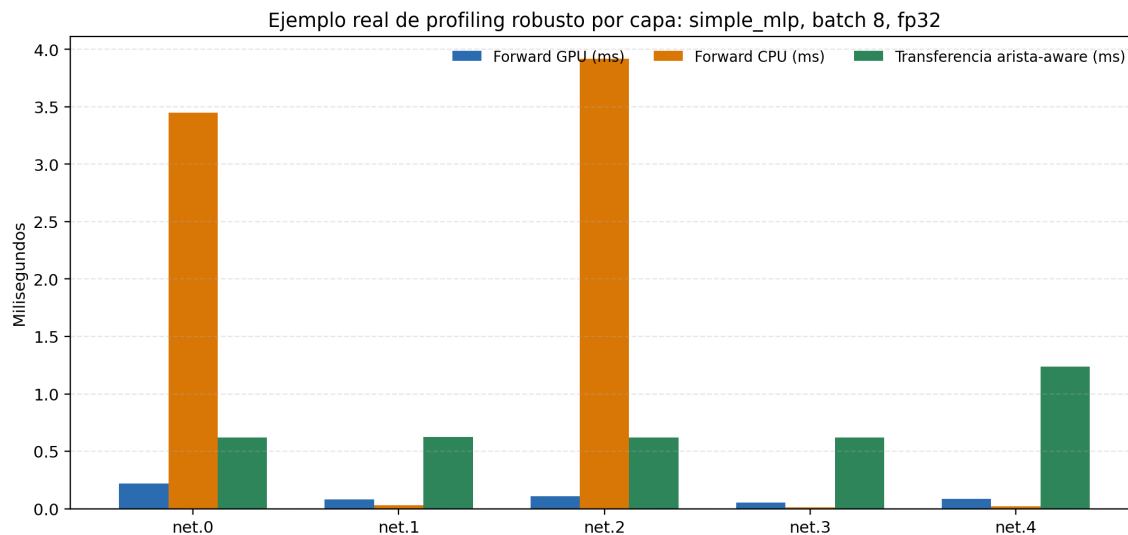


Figura 4.1: Perfil empirico real por capa para `simple_mlp` en smoke con batch 8 y precision `fp32`. La figura se genera directamente a partir de `simple_mlp_metrics_stats.csv`.

Desde una perspectiva doctoral, este punto debe leerse con una cautela adicional. El profiling no es valioso solo porque mida, sino porque decide que cuenta como medida metodologicamente admisible. Una corrida aislada, desprovista de contexto numerico, precision efectiva o procedencia estructural, puede ser suficiente para depurar software; no lo es para sustentar una conclusion de tesis. El regimen inferencial del capitulo exige precisamente lo contrario: que cada coeficiente que alimenta el ILP sea el resultado de una secuencia de decisiones experimentales explicitadas y recuperables.

La Figura 4.1 muestra un ejemplo real de esa instrumentacion sobre `simple_mlp`. En lugar de resumir el modelo completo en una unica latencia agregada, el artefacto separa costo forward en GPU, costo forward en CPU y penalizacion de transferencia consciente de arista para cada capa. Lo metodologicamente importante no es solo que existan diferencias absolutas entre capas, sino que el perfil resultante exhibe heterogeneidad interna suficiente para justificar una politica de asignacion selectiva en lugar de una colocacion monolitica.

La Tabla 4.2 deja ver otra propiedad relevante: el problema no se reduce a escoger siempre GPU o siempre CPU. Las capas `net.0` y `net.2` muestran ventaja de tiempo en GPU pero con una penalizacion de transferencia comparable, mientras que `net.4` introduce una frontera de salida con costo comunicativo relativamente alto. Esta estructura de compromiso es precisamente la que vuelve razonable el paso desde profiling empirico hacia optimizacion declarativa.

Cuadro 4.1: Contrato minimo de artefactos del pipeline de profiling.

Artefacto	Nivel	Rol metodologico
*_metrics.csv	capa-corrida	Medicion cruda multicanal
*_meta.json	corrida	Contexto de hardware y politicas
*_graph_nodes.csv	estructura	Nodos con huellas de computo
*_graph_edges.csv	estructura	Dependencias y topologia util
*_transfer_edges.csv	arista	Costos calibrados de comunicacion
*_metrics_stats.csv	agregado	Coefficientes robustos para ILP

Cuadro 4.2: Extracto empirico de coeficientes robustos de profiling para `simple_mlp`.

Capa	GPU fwd (ms)	CPU fwd (ms)	Transferencia (ms)	Memoria GPU (MB)	Calidad
net.0	0.220	3.448	0.622	19.335	alta dispersion
net.2	0.110	3.918	0.621	19.343	alta dispersion
net.4	0.088	0.022	1.235	19.343	alta dispersion

4.9. 4.9 Del dato observado al coeficiente de decision

Una contribucion metodologica menos visible, pero crucial, de este capitulo consiste en establecer la transformacion legitima entre observacion cruda y coeficiente de decision. El punto de partida no es una ecuacion cerrada, sino una nube de eventos con ruido, variabilidad de replica y dependencias contextuales. El punto de llegada no es una estadistica descriptiva neutra, sino un parametro que condiciona una decision binaria ejecutable. Entre ambos extremos media una cadena de validacion que incluye limpieza, agregacion, chequeo de dispersion, verificacion de precision y asociacion estructural con el grafo.

Este pasaje importa porque, en ausencia de dicha cadena, la optimizacion posterior quedaria epistemologicamente suspendida: seria imposible determinar si la solucion ILP mejora realmente la politica de asignacion o simplemente explota artefactos de medicion mal controlados. La tesis evita ese riesgo al convertir el profiling en una etapa con semantica inferencial propia. La decision combinatoria deja de apoyarse en numeros meramente disponibles y pasa a sostenerse en numeros metodologicamente justificados. La diferencia entre medicion disponible y coeficiente defendible es, de hecho, una de las lecciones que separa un benchmark utilitario de un pipeline de decision comparable con propuestas de colocacion automatica de mayor ambicion metodologica [5, 4].

Nota de borrador: Para la version de deposito conviene anadir aun dos figuras complementarias: la calibracion completa del modelo alpha-beta y una distribucion de replicas por canal para la campana multiservidor.

Cuadro 4.3: Cadena de transformacion desde observacion cruda hasta coeficiente util para optimizacion.

Etapa	Funcion inferencial
Medicion cruda por replica	Captura eventos, tiempos, energia y memoria bajo una configuracion concreta y versionable.
Control de validez	Verifica precision efectiva, soporte de hardware, semillas y consistencia basica de ejecucion.
Agregacion robusta	Resume tendencia central y dispersion sin colapsar la heterogeneidad experimental relevante.
Asociacion estructural	Vincula cada metrica con nodos y aristas del grafo computacional admisible.
Coeficiente para ILP	Entrega un escalar interpretable y trazable que puede entrar en funcion objetivo o restricciones.

Capítulo 5

Formulacion ILP Robusta para Particion Heterogenea CPU-GPU

5.1. 5.1 Introduccion y alcance del modelo (congelado en Fase 0)

La formulacion ILP presentada en este capitulo transforma en decision combinatoria la evidencia empirica construida en el Capitulo 4. El problema no se define sobre costos ideales, sino sobre coeficientes medidos, robustificados y sometidos a criterios de admisibilidad metodologica. Por ello, la principal virtud del modelo no es solo su correccion algebraica, sino la continuidad que preserva entre fenomenologia de runtime y politica ejecutable. Esta continuidad marca una diferencia importante frente a lineas de trabajo donde la colocacion se aprende o se heuristiza sin un contrato tan estricto entre medicion, formulacion y validacion [5, 4, 11].

En coherencia con el cierre de Fase 0, el alcance del modelo debe leerse desde la formulacion fuerte comprometida por la tesis: soporte para separacion forward/backward, persistencia de activaciones y compatibilidad con scheduling asincrono. Las ecuaciones base descritas aqui fijan el nucleo de asignacion binaria y costo de corte; las extensiones posteriores muestran como ese nucleo se integra dentro de un sistema mas amplio sin perder interpretabilidad.

5.2. 5.2 Notacion formal

Sea $G = (V, E)$ un grafo dirigido aciclico donde cada nodo representa una unidad de computo asignable y cada arista representa una dependencia tensorial potencialmente costosa si se corta entre dispositivos. Para cada nodo se dispone de costos robustos de tiempo, energia y memoria en CPU y GPU; para cada arista se dispone de un costo de transferencia derivado del pipeline de profiling. Esta notacion no surge de una convencion abstracta, sino del contrato de artefactos que enlaza `*_metrics_stats.csv`, `*_graph_edges.csv` y `*_transfer_edges.csv` con el constructor de la instancia ILP.

La ventaja de fijar desde el principio esta correspondencia entre simbolos y artefactos es metodologica. Permite que cada termino del modelo posea una interpretacion operacional y una ruta de verificacion directa en el repositorio. En otras palabras, la notacion formal no reemplaza la trazabilidad; la organiza.

De forma mas explicita, para cada nodo $v \in V$ se consideran coeficientes robustos T_v^{gpu} , T_v^{cpu} para tiempo, E_v^{gpu} , E_v^{cpu} para energia y M_v^{gpu} , M_v^{cpu} para memoria. Para cada arista $(u, v) \in E$ se define un costo de corte C_{uv}^{tr} que resume latencia y volumen de transferencia bajo la calibracion disponible. Adicionalmente, se introducen presupuestos B_{gpu} y B_{cpu} y, en la version extendida del modelo, familias de variables auxiliares para persistencia, re-computacion y decisiones desacopladas de forward y backward. Esta explicitacion conviene porque aclara que el ILP no opera sobre magnitudes abstractas, sino sobre un conjunto de observables transformados en coeficientes de decision.

5.3. 5.3 Variables de decision

La variable binaria de asignacion x_v expresa si un nodo se ejecuta en GPU o en CPU, mientras que la variable y_{uv} indica si la arista (u, v) constituye un corte inter-dispositivo. Esta pareja de variables sintetiza la intuicion fisica del problema: toda ganancia local por llevar una capa a GPU puede quedar anulada si dicha decision fragmenta en exceso el grafo e induce comunicacion recurrente.

La inclusion explicita de variables de corte cumple ademas una funcion explicativa en la tesis. Permite descomponer el objetivo total en costo nodal y costo comunicativo, y con ello discutir causalmente por que una solucion supera o no a un baseline greedy. Sin esa explicitud, la optimizacion correria el riesgo de convertirse en una caja negra de asignaciones binarias poco interpretables.

5.4. 5.4 Funcion objetivo multicriterio

La funcion objetivo escalariza tiempo, energia y transferencia mediante pesos w_t , w_e y w_{tr} . La eleccion de esta forma lineal responde a una razon practica y a una razon epistemica. La practica es que mantiene el problema dentro de una clase de resolucion madura y bien entendida. La epistemica es que obliga a declarar con claridad la politica experimental: todo resultado reportado en este capitulo depende de una ponderacion explicita y, por tanto, debe interpretarse como respuesta condicionada a un regimen de preferencias metodologicamente documentado.

Esta escalarizacion no pretende negar la naturaleza multicriterio del problema, sino ofrecer un mecanismo auditable para recorrer regiones del espacio de compromiso. Los barridos sobre presupuestos GPU y los estudios de sensibilidad posteriores cumplen precisamente la funcion de mostrar hasta que punto la solucion cambia cuando cambian los pesos o la factibilidad fisica.

En su forma base, la funcion objetivo puede escribirse como

$$\text{mín } Z = w_t \sum_{v \in V} [x_v T_v^{gpu} + (1 - x_v) T_v^{cpu}] + w_e \sum_{v \in V} [x_v E_v^{gpu} + (1 - x_v) E_v^{cpu}] + w_{tr} \sum_{(u,v) \in E} y_{uv} C_{uv}^{tr}.$$

Esta expresion deja ver una hipotesis central del modelo: el costo total se descompone en contribuciones nodales y contribuciones de corte. La linealidad no afirma que el hardware sea realmente lineal en todos sus efectos, sino que la aproximacion adoptada es suficientemente interpretable y operativa para sostener una politica de decision auditable. La tesis asume esa simplificacion de forma explicita para poder discutir luego, con rigor, donde estan sus limites y que fenomenos quedan fuera de la envolvente del modelo.

5.5. 5.5 Linealizaciones exactas y restricciones

La linealizacion exacta de la variable de corte mediante desigualdades sobre x_u , x_v e y_{uv} preserva fidelidad semantica sin abandonar el dominio MILP. Esto es importante porque evita recurrir a aproximaciones que podrian abaratar artificialmente ciertos cortes o alterar la logica del problema. Las restricciones de memoria por dispositivo, por su parte, convierten la solucion en una politica fisicamente viable y no solo en una mejora aritmetica sobre papel.

Desde el punto de vista de tesis, la relevancia de estas restricciones va mas alla de su

contenido tecnico. Cada una representa una salvaguarda contra conclusiones vacias: sin linealizacion correcta no hay semantica fiable del corte; sin presupuestos de memoria no hay viabilidad operativa; sin dominio binario no hay puente directo hacia el runtime.

Una formulacion minima de estas condiciones puede expresarse como

$$y_{uv} \geq x_u - x_v, \quad y_{uv} \geq x_v - x_u,$$

acompanada por

$$y_{uv} \leq x_u + x_v, \quad y_{uv} \leq 2 - (x_u + x_v),$$

de modo que y_{uv} capture exactamente la discrepancia binaria entre extremos de la arista. A ello se anaden restricciones de memoria del tipo

$$\sum_{v \in V} x_v M_v^{gpu} \leq B_{gpu}, \quad \sum_{v \in V} (1 - x_v) M_v^{cpu} \leq B_{cpu},$$

que convierten la factibilidad en una condicion estructural del problema. En la version extendida, estas desigualdades se enriquecen con variables asociadas a retencion y re-computacion, pero la idea permanece: la solucion debe ser simultaneamente interpretable, resoluble y ejecutable.

5.6. 5.6 Robustificacion estadistica

La robustificacion mediante $\mu + k_\sigma \sigma$ expresa de forma compacta una decision metodologica profunda: las politicas de particion no deben optimizar solo el promedio, sino una version del costo que incorpora incertidumbre observada. Este desplazamiento es especialmente valioso en infraestructura heterogenea, donde el ruido por replica, la variabilidad termica o los efectos de scheduler pueden volver fragil una solucion ajustada al promedio puntual. En terminos conceptuales, la eleccion se emparenta con la intuicion que subyace a los esquemas clasicos de checkpointing y reduccion de memoria: no basta con un costo esperado atractivo si la politica resultante se apoya en un regimen experimental inestable [2, 1].

En el proyecto, el parametro k_σ se ha estudiado con barridos controlados. Sobre el caso smoke de `simple_mlp`, el valor base $k_\sigma = 1,0$ produce un objetivo ILP de 1.810442 a 64 MB de presupuesto GPU; reducirlo a 0.0 genera una solucion mas optimista con objetivo 1.310637, mientras incrementarlo a 2.0 eleva el objetivo a 2.310246. Este comportamiento

confirma que k_σ opera efectivamente como un mando de conservadurismo controlado.

La interpretacion de esta robustificacion debe explicitarse con cuidado. El modelo no pretende construir una teoria probabilistica completa de la incertidumbre, sino introducir una proteccion operacional contra soluciones demasiado sensibles al promedio puntual. En ese sentido, k_σ funciona como un parametro de aversion al riesgo experimental: cuanto mayor es su valor, mayor es la penalizacion que la formulacion impone a coeficientes observados con alta dispersion. La tesis considera esta operacion como una decision metodologica defendible porque traslada al modelo una propiedad real del dato: su estabilidad observacional.

5.7. 5.7 Integracion multi-hardware

Cuando se dispone de perfiles equivalentes provenientes de varios hosts, la tesis propone dos politicas de fusion: peor caso por coeficiente y media mas dispersion inter-host. La primera prioriza seguridad operacional; la segunda busca equilibrio entre rendimiento promedio y heterogeneidad del parque. En ambos casos, la agregacion multi-hardware se concibe como un mecanismo de transferibilidad, no como una etapa cosmetica de posprocesado.

Este punto es central para el cierre doctoral pendiente. El repositorio ya contiene runbooks y protocolo metodologico para construir el dataset multi-servidor; lo que falta es ejecutar esa campaña con suficiencia estadistica. Por tanto, la integracion multi-hardware debe presentarse en este borrador como capacidad ya dispuesta metodologicamente, pero aun en proceso de consolidacion empirica plena.

5.8. 5.8 Complejidad y trazabilidad

Si el grafo tiene n nodos y m aristas, el modelo introduce $n + m$ variables binarias y un numero de restricciones dominado por las linealizaciones de corte. Esta observacion importa porque delimita la escalabilidad practica del enfoque y justifica la existencia de backends diferenciados: ‘pulp’ con CBC para resolucion general, ‘exhaustive’ para subinstancias pequenas usadas como oraculo metodologico y ‘auto’ para seleccion pragmatica de backend.

La trazabilidad del solver se refuerza mediante artefactos de salida estructurados, entre ellos `ilp_assignment.csv`, `ilp_cut_edges.csv` e `ilp_solution_summary.json`. Esta tripleta permite auditar la solucion desde tres perspectivas: politica por nodo, cortes inducidos y resumen global de factibilidad y objetivo. En una tesis aplicada, tal

descomposicion reduce la distancia entre ecuacion y resultado observable.

Conviene anadir que la complejidad computacional del problema no se invoca aqui como argumento de autoridad, sino como limite practico que orienta el disenio experimental. Saber que la formulacion crece con el numero de nodos y aristas permite entender por que el manuscrito combina diferentes backends y por que algunas campañas se ejecutan en regimen smoke o controlado antes de ampliarse. La tesis no oculta esta restriccion; la incorpora como parte de la honestidad metodologica del modelo.

5.9. 5.9 Supuestos de modelado y frontera de aplicabilidad

Todo modelo declarativo fuerte descansa sobre supuestos que deben hacerse visibles. En esta tesis destacan al menos cinco. Primero, se asume que el grafo disponible conserva suficiente fidelidad estructural para que los cortes tengan significado fisico. Segundo, se asume que los coeficientes robustificados son una aproximacion razonable del costo efectivo por nodo y por arista bajo el regimen experimental considerado. Tercero, se asume que la escalarizacion lineal es adecuada para explorar compromisos entre tiempo, energia y transferencia sin ocultar por completo su semantica. Cuarto, se asume que las restricciones de memoria sintetizan de manera util el principal cuello de botella de factibilidad. Quinto, se asume que el paso desde solucion a plan puede preservarse sin redefinir silenciosamente la politica entre optimizacion, simulacion y runtime.

Ninguno de estos supuestos es inocente. Cada uno fija una frontera de aplicabilidad del modelo y, por tanto, un limite de generalizacion de sus resultados. La fuerza del manuscrito no consiste en negar esa dependencia, sino en explicitarla y someterla a validacion cruzada por sensibilidad, ablacion y contraste operacional. Ese es precisamente el rasgo que distingue una formulacion doctoral de una heuristica utilitaria: no solo produce soluciones, sino que declara bajo que hipotesis esas soluciones deben tomarse en serio.

5.10. 5.10 Validacion comparativa contra baselines (habilitada por Fase 1)

La validacion comparativa frente a baselines cumple una funcion decisiva: distinguir si la mejora procede realmente de la estructura del modelo o de una ventaja trivial facilmente alcanzable sin ILP. En el conjunto smoke consolidado de Fase 1, el mejor punto factible

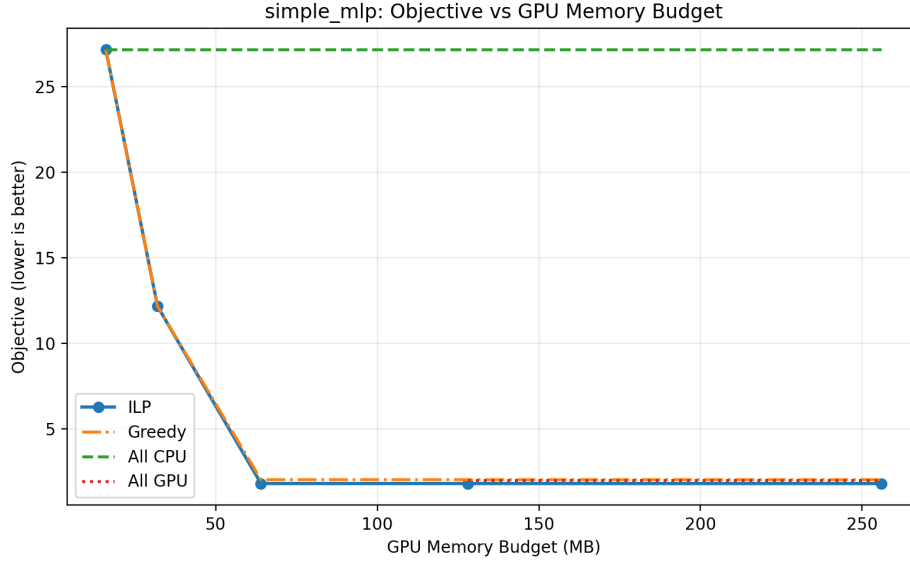


Figura 5.1: Frontera de compromiso objetivo-presupuesto GPU para el caso smoke consolidado de `simple_mlp`.

Cuadro 5.1: Mejor resultado ILP factible por modelo bajo los presupuestos GPU evaluados en el conjunto smoke.

Modelo	Budget GPU	Obj. ILP	Obj. CPU	Obj. Greedy	Mejora vs CPU (%)	Mejora vs Greedy (%)	Capas GPU	Cortes
<code>simple_mlp</code>	64	1.810	27.153	2.035	93.33	11.06	3	1

para `simple_mlp` a 64 MB muestra un objetivo ILP de 1.810 frente a 27.153 para `all_cpu` y 2.035 para `greedy`, lo que equivale a una mejora del 93.33 % respecto a `all_cpu` y del 11.06 % respecto a `greedy`.

Este resultado no debe sobredimensionarse: corresponde a una malla reducida y a un modelo piloto. Sin embargo, si constituye evidencia valida de que la formulacion, incluso en regimen smoke, ya supera tanto a una politica trivial como a una heuristica nodal. Ese hecho justifica la expansion posterior hacia campañas mas amplias y hacia comparativas sobre hardware fusionado.

5.11. 5.11 Estudios de sensibilidad (habilitados por Fase 1)

Los estudios de sensibilidad muestran que la formulacion no solo produce una solucion puntual, sino una respuesta interpretable ante perturbaciones controladas de sus parametros. En el conjunto smoke, el barrido sobre k_σ exhibe una variacion aproximadamente

Cuadro 5.2: Barrido Pareto ILP en el conjunto smoke con comparacion frente a baselines.

Budget GPU	Obj. ILP	Obj. Greedy	Obj. CPU	Mejora vs CPU (%)	Mejora vs Greedy (%)	Cortes
16	27.153	27.153	27.153	0.00	0.00	0
32	12.163	12.163	27.153	55.20	0.00	1
64	1.810	2.035	27.153	93.33	11.06	1
128	1.810	2.035	27.153	93.33	11.06	1
256	1.810	2.035	27.153	93.33	11.06	1

lineal del objetivo en torno al punto base, mientras que el barrido sobre $w_{transfer}$ revela un comportamiento monotono pero no lineal: cuando $w_{transfer} = 0,0$, el modelo tolera tres cortes y reordena capas hacia una solucion mas agresiva; al aumentar el peso de transferencia, la politica reduce fragmentacion y encarece el objetivo total bajo una lectura mas conservadora del sistema.

Esta informacion es de interes doctoral porque permite afirmar que los parametros del modelo tienen semantica economica verificable. No son perillas arbitrarias, sino mecanismos de politica cuyo efecto sobre la solucion puede estudiarse con trazas reproducibles.

5.12. 5.12 Ablaciones formales (habilidades por Fase 1)

La suite de ablaciones cumple la funcion de desagregar el aporte de los componentes metodologicos del modelo. En el resumen disponible para `simple_mlp`, la variante completa obtiene objetivo 1.810442; eliminar la robustificacion reduce el objetivo en 0.499805 unidades, reflejando un escenario optimista irreal; eliminar topologia o costos de transferencia reduce el objetivo en 0.438429 unidades y altera la estructura de la particion, pasando de un plan con un corte y tres capas en GPU a variantes mas fragmentadas o semantica empobrecida.

Estas ablaciones son importantes porque muestran que la mejora no proviene solo de un solver aplicado sobre cualquier costo disponible. Proviene de la conjuncion entre grafo admisible, penalizacion comunicativa y robustificacion estadistica. En ausencia de alguno de esos componentes, el problema resuelto cambia de significado.

5.13. 5.13 Extension avanzada: persistencia de activaciones y rematerializacion (Fase 4)

La extension avanzada deja de ser prospectiva en el estado actual del proyecto. La Fase 4 ha integrado decision dual forward/backward, persistencia de activaciones, recomputacion, checkpointing y compatibilidad con prefetching en `src/ilp/advanced_terms.py`, `src/ilp/model_builder.py` y en el runtime asociado. Esto permite afirmar que la tesis no se cierra sobre un ILP minimamente viable, sino sobre una version extendida cuyo significado operacional ya fue contrastado en evidencia controlada. La articulacion entre persistencia, recomputacion y particion heterogenea enlaza directamente con la tradicion inaugurada por Revolve y retomada despues por estrategias de memoria sublineal y offloading, aunque aqui se reinterpreta como decision conjunta dentro de una formulacion declarativa unificada [2, 1, 9].

No obstante, desde el punto de vista de escritura doctoral conviene mantener una distincion clara entre dos niveles de evidencia. Existe ya evidencia operacional de que la extension funciona y puede ejecutarse sobre un caso dual controlado; lo que aun falta es una explotacion experimental amplia de esa extension sobre una matriz final de modelos y hosts. El manuscrito debe sostener ambas verdades sin confundir madurez de implementacion con amplitud de validacion empirica.

5.14. 5.14 Amenazas a la validez

Las amenazas a la validez del modelo ILP se agrupan en tres clases. La primera es la amenaza de medicion: coeficientes sesgados por ruido, precision degradada o muestra insuficiente pueden inducir optimizaciones sobre un problema mal especificado. La segunda es la amenaza de modelado: la linealidad no captura todas las interacciones no lineales del hardware, como saturacion de bus o contencion compartida. La tercera es la amenaza de transferibilidad: una politica afinada sobre un host unico puede no generalizar a un parque heterogeneo.

Las mitigaciones ya integradas en el proyecto son sustantivas: compuertas de admisibilidad sobre calidad muestral y procedencia estructural, barridos de sensibilidad, ablaciones, comparacion con baselines y capacidad de fusion multi-hardware. Aun asi, el cierre doctoral pleno exige ejecutar el protocolo multi-servidor y ampliar la base de evidencia sobre calidad final del entrenamiento. La necesidad de este cierre adicional no contradice la validez del modelo; la situa, con honestidad, en el mismo horizonte comparativo en el que la

Cuadro 5.3: Amenazas a la validez del modelo ILP y estado de mitigacion.

Amenaza	Mitigacion ya integrada	Evidencia pendiente de cierre
Medicion sesgada o inestable	Profiling con replicas, robustificacion estadistica, control de precision efectiva y banderas de calidad	Campana multiservidor con base estadistica ampliada
Modelo lineal insuficiente	Sensibilidad, ablaciones y contraste posterior con simulacion y runtime	Calibraciones no lineales y estudios de saturacion a mayor escala
Falta de transferibilidad entre hosts	Diseno metodologico para fusion multi-hardware y protocolo de validacion transversal	Consolidacion empirica del dataset inter-host y su explotacion final
Brecha entre solver y ejecucion real	Representacion formal de plan, simulador y runtime con trazabilidad de procedencia	Matriz mas amplia de contraste prediccion-observacion

literatura reciente ha tenido que conciliar capacidad de optimizacion con diversidad real de plataformas [8, 9, 11].

Visto desde una perspectiva mas amplia, este capitulo cumple una funcion que trasciende la simple obtencion de una solucion optima. Su papel es mostrar que la asignacion heterogenea puede formularse como decision cientificamente discutible, y no solo como una heuristica afortunada. El ILP obliga a explicitar hipotesis de costo, restricciones fisicas y relaciones entre nodos y cortes. Esa explicitud es una ganancia epistemologica en si misma, porque permite que el lector cuestione y refine el modelo sin perder de vista que toda modificacion tiene consecuencias operacionales precisas.

La Tabla 5.3 deja ver que las amenazas a la validez no son un apartado defensivo secundario, sino una parte constitutiva del sentido doctoral del capitulo. Explicitar que esta mitigado, que permanece abierto y bajo que protocolo se resolvera es una forma de honestidad cientifica, pero tambien una forma de fortalecer la interpretabilidad de la solucion obtenida.

Capítulo 6

Ejecucion Hibrida, Simulacion y Validacion Experimental

6.1. 6.1 Introduccion: cierre del bucle planificacion-ejecucion

Este capitulo cierra la brecha historicamente mas delicada del proyecto: la distancia entre una solucion ILP valida en papel y una politica cuya materializacion pueda observarse en runtime. En una tesis de sistemas, ese salto no puede dejarse implicito. Una politica de particion solo adquiere valor cientifico pleno cuando es posible representarla como plan, validarla por simulacion y contrastarla en ejecucion real con metadatos suficientes para interpretar divergencias. En ese sentido, este capitulo prolonga la logica del Capitulo 5: ya no pregunta solo que plan es mejor, sino bajo que condiciones una recomendacion optimizada conserva significado cuando entra en contacto con la semantica real del framework y del hardware [7, 3, 6].

La arquitectura metodologica del proyecto aborda este problema en tres olas. La primera convierte la solucion ILP en una representacion formal de plan. La segunda la somete a simulacion determinista, verificando integridad topologica, cortes y presupuestos. La tercera la ejecuta sobre un runtime hibrido capaz de registrar latencia, memoria, energia, transferencias y procedencia del plan seleccionado.

6.2. 6.2 Representacion formal del plan de ejecucion (Fase 2)

La representacion formal del plan, soportada por los modulos `runtime/plan_representation.py` y `runtime/device_plan.py`, traduce la salida del solver en una estructura consumible por simulador y ejecutor. Dicha estructura incluye asignacion de nodos a dispositivos, cortes esperados, presupuestos observados y, en el pipeline endurecido, metadatos adicionales sobre modo de seleccion del plan, fuente de procedencia, presupuesto GPU y objetivo asociado.

Este detalle aparentemente administrativo tiene relevancia metodologica. Gracias a esos metadatos, el proyecto puede distinguir si la validacion hibrida se hizo sobre una solucion unrestricted o sobre el mejor punto Pareto presupuestado. La correccion reciente del pipeline para privilegiar `pareto_best` refuerza precisamente esta trazabilidad y evita ambigüedades en la narrativa experimental.

6.3. 6.3 Simulador hibrido: arquitectura y validacion topologica (Fase 2)

El simulador del modulo `runtime/simulator.py` reinterpreta el plan sobre el grafo de dependencias y produce costos previstos de tiempo, energia, memoria y comunicacion. Su funcion no es reemplazar la ejecucion fisica, sino actuar como filtro previo de coherencia y como instrumento de comparacion posterior. Si un plan viola presupuestos, carece de costos de transferencia en modo estricto o incurre en inconsistencias topologicas, el simulador lo detecta antes de consumir recursos experimentales en runtime.

La Fase 2 del proyecto quedo formalmente cerrada cuando este componente demostro estabilidad bit a bit sobre entradas validas y capacidad de rechazo explicito sobre entradas inconsistentes. Esa clausura otorga al simulador un estatus importante en la tesis: no es una idea futura, sino una pieza ya operativa y citable como evidencia pre-ejecucion.

6.4. 6.4 Estudio de prediccion: latencia, memoria y costos de transferencia (Fase 2)

La utilidad principal del simulador reside en descomponer el costo esperado antes de la ejecucion fisica. Sobre un plan dado, puede reportar latencia total estimada, memoria por

dispositivo, energia agregada y cantidad esperada de cortes, junto con desgloses CSV y resumentes JSON. Esta capacidad es fundamental para una tesis doctoral porque permite discutir no solo si una politica es mejor, sino por que se espera que lo sea antes de observarla en hardware.

En el estado actual del proyecto, la evidencia controlada de Fase 4 ya ofrece un ejemplo claro: para un caso dual sobre `simple_mlp`, el simulador valida topologia y costos del plan extendido y estima un objetivo robusto de 19.884783, un tiempo total de 19.547401 ms y dos cortes efectivos. Aunque se trata de un caso focal, demuestra que la representacion extendida no es solo algebraicamente admisible, sino predictivamente util.

6.5. 6.5 Implementacion del ejecutor hibrido en PyTorch (Fase 3)

El ejecutor hibrido implementado en `src/runtime/hybrid_executor.py` materializa la politica de asignacion sobre entrenamiento real en PyTorch. Su interfaz experimental, `validation/run_hybrid_execution.py`, aplica un plan sobre un modelo y registra trazas observadas de forward, backward, optimizacion, transferencias, memoria maxima y energia, con soporte adicional para procedencia del plan y para fuentes reales de dataset. Esta instrumentacion convierte la ejecucion runtime en un experimento interpretado y no en una simple demostracion funcional: cada corrida deja una huella suficientemente rica para reconstituir por que una politica se comporto como lo hizo. La decision de apoyarse en PyTorch no es contingente: aprovecha un ecosistema con semantica dinamica bien establecida, pero lo somete a un regimen de trazabilidad mas severo que el habitual en librerias de entrenamiento de proposito general [7].

La Fase 3 quedo formalmente cerrada al validarse en hardware CPU-GPU del entorno de desarrollo una ejecucion guiada por plan ILP para `simple_mlp`, incluyendo exportacion de energia observada y eventos de cruce CPU-GPU. Posteriormente, la Fase 4 extendio este soporte a casos duales forward/backward, persistencia de activaciones y prefetching, lo que fortalece el valor del runtime como pieza central de la contribucion doctoral.

6.6. 6.6 Baselines reales y comparativos experimentales (Fase 3)

La comparacion frente a `all_cpu`, `all_gpu`, `greedy` e `ilp_plan` resulta metodologicamente obligatoria. Solo asi puede distinguirse si la mejora observada en runtime proviene de la estructura de la politica o de una configuracion trivial. El proyecto ya dispone de esta comparacion en el plano ILP y ha comenzado a consolidarla en ejecucion hibrida mediante artefactos observacionales que incluyen metrica final de calidad cuando el modelo y el dataset lo permiten. La comparativa es ademas coherente con el estado del arte reciente: tanto las propuestas de pipeline como los sistemas de paralelizacion automatizada solo resultan interpretables cuando se sitúan frente a extremos triviales y frente a heurísticas competitivas bajo un contrato experimental comun [3, 6, 11].

En esta etapa, la tesis debe ser rigurosa con el alcance de la evidencia. Ya existe validacion operacional suficiente para afirmar que el runtime ejecuta planes ILP y extensiones duales de manera auditada; sin embargo, la comparacion sistematica amplia entre todos los baselines y sobre varias familias de modelo sigue siendo trabajo de cierre de Fase 5.

Una lectura doctoral mas exigente obliga ademas a distinguir entre comparacion de rendimiento y comparacion de significado. Un baseline no sirve solo para saber si una politica es mas rapida; sirve para revelar que tipo de inteligencia decisional aporta realmente el modelo. Si `all_cpu` y `all_gpu` representan extremos triviales de colocacion, y `greedy` representa una heuristica local, entonces la superioridad del plan ILP debe interpretarse como evidencia de que el problema contiene interdependencias globales que no pueden resolverse con decisiones puramente miopes. Esta es, en ultima instancia, la razon teorica por la cual la tesis insiste en mantener los baselines dentro del mismo contrato experimental.

6.7. 6.7 Prediccion frente a observacion: contraste operacional (Fase 3)

El contraste entre prediccion y observacion es uno de los elementos que mas elevan el estandar cientifico del trabajo. Si el simulador y el runtime convergen razonablemente en tendencias y orden relativo de politicas, la tesis gana en credibilidad causal. Si divergen, la discrepancia deja de ser un problema oculto y se convierte en una fuente de conocimiento sobre aspectos no capturados del sistema, como overhead de framework, solapamiento

Cuadro 6.1: Comparacion prediccion-observacion en el caso dual controlado de `simple_mlp`.

Metrica	Predicho	Observado	Error relativo
Latencia total (ms)	19.547	147.638	0.868
Energia total (J)	0.145	1.704	0.915
Memoria GPU pico (MB)	19.335	10.698	0.807
Eventos de transferencia	2	2	0.000

imperfecto o contencion de memoria.

Una forma compacta de expresar este contraste consiste en definir, para una metrica observable m , un error relativo de prediccion

$$\varepsilon_m = \frac{|\hat{m} - m^{obs}|}{\text{máx}(m^{obs}, \epsilon)},$$

donde $\hat{m}$ es el valor predicho por el simulador, m^{obs} el valor observado en runtime y ϵ una constante pequena de estabilizacion numerica. Esta definicion no pretende imponer una metrica unica de aceptacion, pero si fijar un lenguaje analitico claro para la comparacion entre politica prevista y politica observada.

El proyecto ya posee la infraestructura para realizar este contraste de manera no ambigua. La seleccion del plan `pareto_best`, la exportacion de metadatos de procedencia y la consolidacion de resumentes por configuracion permiten construir una comparacion limpia entre lo previsto y lo observado. Lo pendiente es ampliar esta comparacion desde casos focales a una matriz final mas extensa y estadisticamente suficiente.

La Figura 6.1 y la Tabla 6.1 incorporan ya ese contraste para el caso controlado dual de `simple_mlp`. El resultado es metodologicamente util por una razon precisa: muestra simultaneamente convergencia estructural y divergencia cuantitativa. El simulador predice correctamente un plan con dos cortes y una ocupacion GPU acotada, pero subestima con claridad la latencia y la energia finales porque no modela de manera exhaustiva el overhead del framework, la gestion dinamica del runtime y otros costos de materializacion. Esa discrepancia no invalida el simulador; delimita su regimen de validez y evita atribuirle un poder predictivo que aun no se ha demostrado.

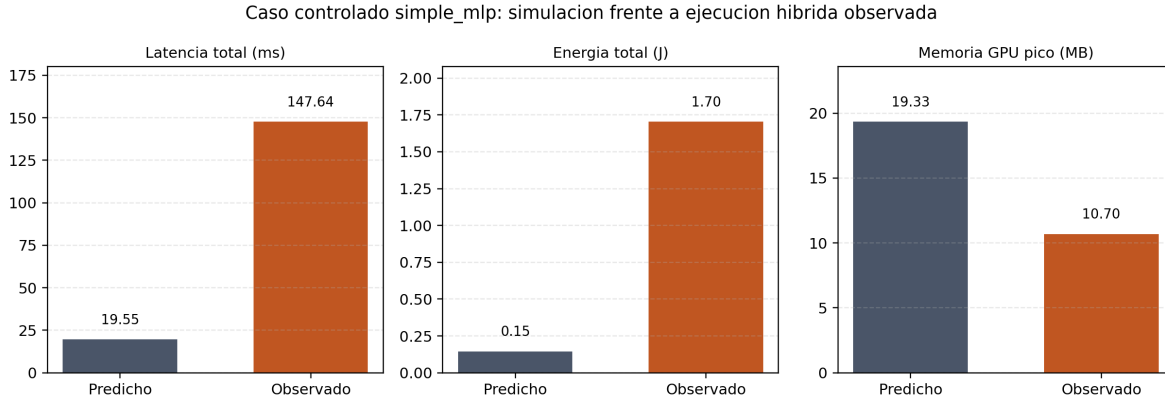


Figura 6.1: Contraste empirico entre simulacion y ejecucion hibrida observada para el caso dual controlado de `simple_mlp`. La figura se genera directamente a partir de los resúmenes JSON de simulacion y runtime.

6.8. 6.8 Exactitud final del modelo y estabilidad de convergencia (Fase 3-5)

Una tesis doctoral sobre entrenamiento heterogeneo no puede cerrarse solo con costos de ejecucion. Debe verificar tambien que la politica hibrida no degrade de manera inaceptable la calidad final del entrenamiento. En el estado actual del pipeline, esta dimension ya cuenta con soporte operacional: el modo tesis registra `dataset_name`, `dataset_split`, `final_loss`, `quality_metric_name` y `final_quality_metric`, lo que hace posible integrar calidad final en la misma cadena de artefactos que tiempo, energia y memoria.

La situacion actual es favorable pero aun incompleta. Ya no existe la brecha metodologica de operar ordinariamente con batches sinteticos; profiling y runtime pueden apoyarse en datasets reales versionables. Sin embargo, la evidencia consolidada sobre exactitud final sigue siendo parcial por arquitectura y debe ampliarse en la campana doctoral final para convertir esta capacidad operativa en afirmacion generalizable.

Este punto es crucial porque reordena la jerarquia de criterios del manuscrito. La mejora en latencia o memoria solo es cientificamente defendible si no altera de forma inadmisible la semantica del entrenamiento. En otras palabras, la tesis no busca solo una politica de asignacion eficiente, sino una politica de asignacion admisible. La exactitud final, la estabilidad de convergencia y la coherencia del proceso de optimizacion deben leerse, por tanto, como restricciones de aceptacion epistemica tan importantes como las restricciones de memoria o los costos de transferencia.

6.9. 6.9 Extensiones con persistencia de activaciones y scheduling asincrono (Fase 4)

Las extensiones avanzadas ya son parte del sistema operativo del proyecto. La Fase 4 integro persistencia de activaciones, recomputacion, checkpointing, separacion forward/backward y soporte de transferencia asincrona con prefetching explicito. La evidencia controlada sobre `simple_mlp` muestra incluso una recolocacion real de backward para `net.0` y eventos efectivos de prefetch sobre `net.1`, lo que acredita materializacion runtime y no solo disponibilidad de API. En su significado tecnico, estas extensiones acercan el proyecto a la conversacion contemporanea sobre offloading y entrenamiento consciente de memoria, aunque manteniendo una diferencia central: aqui la extension no queda encapsulada como truco local del runtime, sino ligada a un plan formal derivado del modelo declarativo [1, 8, 9].

Este hecho es importante para la escritura del capitulo. Permite abandonar el tono prospectivo que aun aparece en ciertos esqueletos antiguos y reemplazarlo por una narrativa de capacidad implementada con validacion focal. Lo que sigue pendiente ya no es demostrar que la extension existe, sino explotar esa extension sobre una matriz mas amplia y extraer conclusiones comparativas robustas.

6.10. 6.10 Analisis integrado de resultados y limites experimentales (Fase 5)

En conjunto, la evidencia reunida hasta ahora permite sostener tres afirmaciones con base verificable. Primero, el proyecto ya no es un ILP offline desconectado del runtime: dispone de plan formal, simulador y ejecutor hibrido observacional. Segundo, la version endurecida del pipeline evita una ambigüedad metodologica importante al alinear la validacion hibrida con el mejor plan Pareto presupuestado. Tercero, la extension avanzada con persistencia de activaciones y scheduling asincrono ya tiene evidencia operacional controlada. Leido en continuidad con los capitulos previos, el resultado es una cadena argumental mas cerrada: el profiling produce coeficientes, el ILP decide bajo restricciones, y el runtime somete esa decision a contraste material sin perder procedencia ni interpretabilidad.

Al mismo tiempo, persisten limites que la tesis debe explicitar. La mayor brecha restante no es de implementacion sino de evidencia: falta completar una campana multi-servidor y una consolidacion amplia de calidad final y contraste prediccion-observacion por familia

de modelo. Reconocer este punto no debilita la tesis; al contrario, evita inflar el alcance de la evidencia actualmente disponible y orienta con precision el trabajo experimental final.

6.11. 6.11 Significado metodologico del cierre predictivo

El cierre entre simulacion y runtime constituye, probablemente, la contribucion mas delicada del manuscrito desde el punto de vista de filosofia experimental. Muchas propuestas de optimizacion para sistemas heterogeneos se detienen en una solucion offline o en una prediccion agregada de costo. En esos casos, el salto desde la recomendacion matematica hasta la ejecucion real queda desplazado a una zona de supuestos tacitos. La presente tesis insiste en que ese salto debe formar parte de la contribucion y no quedar externalizado.

La razon es simple: una politica de asignacion solo adquiere estatuto cientifico pleno cuando puede ser representada, interpretada, rechazada y observada bajo el mismo contrato de trazabilidad. La simulacion ofrece una forma de razonamiento predictivo; el runtime, una forma de contraste material. Entre ambas se juega la credibilidad final del enfoque. De ahi que este capitulo no deba leerse como un apendice de implementacion, sino como el lugar donde la tesis demuestra que la optimizacion declarativa no es un ejercicio abstracto desligado del comportamiento real del sistema.

Capítulo 7

Conclusiones, Limites y Trabajo Futuro

7.1. 7.1 Sintesis de resultados frente a la hipotesis

La hipotesis central de la tesis afirma que una politica de particion heterogenea CPU-GPU, construida a partir de evidencia empirica robusta y optimizacion declarativa, puede mejorar el comportamiento global frente a politicas simples sin perder validez operacional. A la fecha de este borrador, esa hipotesis ya cuenta con sustento parcial fuerte. El pipeline de profiling existe y produce coeficientes admisibles; el ILP supera a baselines `all_cpu` y `greedy` en el caso smoke consolidado; la solucion puede transformarse en plan, simularse y ejecutarse en runtime; y la version extendida con decisiones duales y prefetching ya ha sido materializada en evidencia controlada.

El resultado mas claro disponible en esta etapa es el cierre del circuito metodologico minimo. La tesis ya puede afirmar que mide, decide, simula y ejecuta. Lo que aun no puede afirmar con igual amplitud es la universalidad de ese comportamiento sobre una matriz multi-servidor y multi-arquitectura estadisticamente amplia. Por ello, la hipotesis debe presentarse en este borrador como fuertemente apoyada en viabilidad metodologica y en evidencia focal, pero pendiente de cierre empirico transversal final.

7.2. 7.2 Contribuciones metodologicas y de sistemas

Las contribuciones del trabajo pueden sintetizarse en cuatro niveles complementarios. Primero, se construye un sistema de profiling por capa que integra tiempo, energia, memoria, transferencia y procedencia estructural del grafo bajo un contrato de artefactos reproducible.

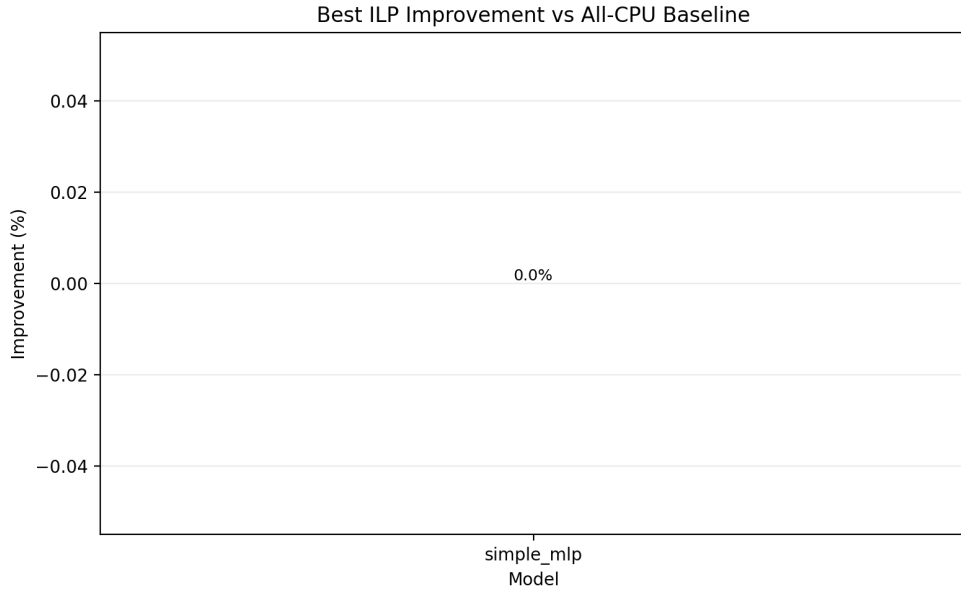


Figura 7.1: Mejora porcentual del mejor plan ILP frente al baseline `all_cpu` en la campaña de Fase 1. Esta figura resume la direccion favorable de la evidencia ILP antes del cierre empirico multi-servidor.

Segundo, se formula un ILP robusto e interpretable que traduce esa evidencia en una decision binaria ejecutable bajo restricciones de memoria y con soporte para sensibilidad, ablaciones y fusion multi-hardware. Tercero, se cierra el puente entre optimizacion y sistemas mediante una representacion formal del plan, un simulador determinista y un ejecutor hibrido observacional. Cuarto, se fija un contrato metodologico que obliga a sostener cada afirmacion con evidencia trazable y que distingue explicitamente entre capacidad implementada, validacion focal y cierre doctoral amplio.

La contribucion original del proyecto no reside solamente en cada pieza aislada, sino en su integracion rigurosa. Ese caracter de sistema completo es el que diferencia esta tesis de propuestas que permanecen en el plano del benchmarking, del modelado offline o del prototipo runtime sin trazabilidad suficiente.

7.3. 7.3 Limites del modelo y del protocolo experimental

El trabajo presenta limites que deben exponerse con precision. En el plano de modelado, la formulacion lineal no capta todas las no linealidades del hardware real, y el modelo

alpha-beta de transferencia sigue siendo una aproximación de primer orden. En el plano empírico, parte de la evidencia disponible procede todavía de mallas smoke o de casos controlados sobre modelos piloto, lo que restringe la generalización inmediata de ciertas conclusiones. En el plano experimental, la principal limitación abierta es la ausencia de una campaña multi-servidor final completamente cerrada con al menos cinco réplicas por configuración en la malla canónica.

Existe además un límite narrativo que la tesis debe evitar: confundir madurez del software con amplitud de la validación. El repositorio ya incorpora capacidades avanzadas como separación forward/backward, persistencia de activaciones y prefetching; sin embargo, su explotación estadística amplia aún está en curso. La conclusión correcta, por tanto, no es que el sistema esté incompleto, sino que su cierre empírico de nivel doctoral todavía requiere una campaña final más extensa.

7.4. 7.4 Líneas de trabajo futuro

Las líneas de trabajo futuro se desprenden con claridad del estado actual del proyecto. La primera y más inmediata es ejecutar la validación multi-servidor definida en el protocolo metodológico del repositorio, consolidando un dataset heterogéneo que permita resolver ILP robusto inter-host y contrastar su transferibilidad real. La segunda es cerrar la comparación sistemática predicción-versus-observación sobre una matriz final de modelos, no solo sobre casos piloto. La tercera es profundizar en la validación de calidad final del entrenamiento, integrando accuracy, loss o métrica específica de tarea como condición de aceptación tan relevante como tiempo o memoria.

Una cuarta línea, más propiamente investigativa, consiste en enriquecer el modelo de costo con términos no lineales o calibraciones por tramos que capturen saturación y contención de bus con mayor fidelidad. Una quinta es ampliar el estudio a familias de modelos más grandes y a regímenes de precisión más diversos, siempre que se preserve el mismo contrato de trazabilidad y admisibilidad metodológica. Estas extensiones no invalidan el núcleo actual del trabajo; lo prolongan hacia una agenda de investigación posterior coherente con los límites ya identificados.

Cuadro 7.1: Trazabilidad entre afirmaciones nucleares del manuscrito y evidencia disponible.

Afirmacion	Evidencia principal en el manuscrito	Alcance actual
El proyecto produce coeficientes empiricos trazables por capa	Capitulo 4, Tabla 4.1, Figura 4.1, Tabla 4.2	Demostrado
La formulacion ILP supera base-lines simples en casos consolidados	Capitulo 5, Figura de objetivo vs presupuesto y tablas de comparacion ILP	Demostrado en campañas focales
La solucion ILP puede transformarse en plan, simularse y ejecutarse	Capitulo 6, secciones de representacion, simulacion y ejecutor hibrido	Demostrado
Existe contraste predictivo en un caso controlado	Capitulo 6, Figura 6.1 y Tabla 6.1	Demostrado en caso focal
Las extensiones duales y de pre-fetching tienen evidencia operacional	Capitulo 6, evidencia controlada de Fase 4 y discusion de extensiones avanzadas	Demostrado en validacion controlada
La tesis ya dispone de cierre empirico transversal multi-servidor	Protocolo multiservidor y discusion de limites en Capítulos 6 y 7	Pendiente

7.5. 7.5 Trazabilidad final entre afirmaciones y evidencia

Una monografia doctoral sobre sistemas no se cierra adecuadamente solo con una enumeracion de aportes; debe explicitar la correspondencia entre cada afirmacion fuerte y la evidencia que la sostiene. La tabla siguiente resume esa relacion y fija, de manera deliberadamente conservadora, el alcance con el que cada resultado puede presentarse en esta etapa del manuscrito.

La funcion de esta tabla no es solo resumir resultados, sino disciplinar la escritura final. Obliga a distinguir entre lo ya demostrado, lo demostrado solo en regimen focal y lo que aun debe completarse antes del deposito definitivo. Esa distincion es parte de la calidad epistemologica del manuscrito y protege la tesis frente a sobreafirmaciones.

7.6. 7.6 Valor doctoral del trabajo en su estado actual

El valor doctoral del manuscrito, en su estado presente, no debe medirse exclusivamente por la cantidad de funcionalidad implementada ni por la longitud del documento, sino por la coherencia alcanzada entre problema, metodo y evidencia. A este nivel, la tesis ya exhibe un rasgo fuerte: ha transformado una intuicion de ingenieria sobre infrautilizacion de CPU y saturacion de memoria de GPU en una cadena argumental completa que incluye fundamento teorico, observacion empirica, formulacion declarativa, simulacion y validacion operacional. Ese encadenamiento no es trivial y constituye, por si mismo, una contribucion metodologica robusta.

Tambien es importante subrayar que el manuscrito ya se encuentra mas alla del estadio de propuesta conceptual. Existe un pipeline que produce artefactos, un modelo que resuelve decisiones no triviales, una semantica de plan que cierra el paso hacia simulacion y runtime, y evidencia focal de que las extensiones mas ambiciosas del alcance fuerte pueden materializarse de forma controlada. En consecuencia, la tesis no esta en una fase de mera promesa investigativa; se encuentra en una fase de consolidacion y ampliacion empirica. Esa diferencia debe quedar clara porque condiciona el modo en que se presentan tanto los aportes como las limitaciones.

Sin embargo, precisamente por esa madurez, el documento tiene la obligacion de sostener un criterio alto de prudencia interpretativa. Una tesis doctoral rigurosa no solo debe demostrar que puede hacer algo, sino delimitar con claridad donde termina la evidencia efectivamente disponible. El cierre multi-servidor, la ampliacion estadistica de la validacion y la consolidacion final de calidad del entrenamiento no son detalles secundarios: son las piezas que convertiran una demostracion fuerte de viabilidad metodologica en una afirmacion generalizable de alcance doctoral pleno. La honestidad con que el manuscrito marca esa frontera constituye una parte esencial de su calidad cientifica.

7.7. 7.7 Cierre final

La pregunta que ha organizado esta investigacion puede formularse de manera simple: es posible redistribuir de forma cientificamente justificada el entrenamiento profundo entre CPU y GPU para aliviar la presion de memoria sin degradar el valor operacional del proceso. La respuesta que el manuscrito ya puede ofrecer es igualmente clara, aunque matizada: si, bajo un regimen metodologico que mida, modele, simule y observe la politica resultante. Esa condicionalidad no debilita la respuesta; la convierte en una tesis verificable y no en una intuicion optimista.

En ultima instancia, la contribucion del trabajo consiste en mostrar que la heterogeneidad computacional puede dejar de ser un trasfondo pasivo del entrenamiento para convertirse en objeto explicito de decision. Esa transformacion conceptual tiene implicaciones que exceden el caso concreto estudiado: invita a pensar el aprendizaje profundo no solo como optimizacion estadistica sobre datos y parametros, sino tambien como organizacion formal de recursos materiales escasos. En esa interseccion entre teoria del entrenamiento, sistemas heterogeneos y metodologia experimental se situa el nucleo intelectual de la monografia.

El cierre definitivo del doctorado exigira, todavia, ampliar y consolidar la evidencia

transversal. Pero aun en su forma actual, el manuscrito ya fija una posicion fuerte: la particion heterogenea CPU-GPU no debe abordarse como una coleccion de ajustes aislados, sino como una politica formal, empiricamente calibrada y operacionalmente contrastable. Ese es el sentido profundo del trabajo y el criterio bajo el cual debe leerse su aportacion.

Bibliografía

- [1] Tianqi Chen, Bing Xu, Chiyuan Zhang, and Carlos Guestrin. Training deep nets with sublinear memory cost, 2016.
- [2] Andreas Griewank and Andrea Walther. Algorithm 799: Revolve: An implementation of checkpointing for the reverse or adjoint mode of computational differentiation. *ACM Transactions on Mathematical Software*, 26(1):19–45, 2000.
- [3] Yanping Huang, Yong Cheng, Ankur Bapna, Orhan Firat, Dehao Chen, Mia Chen, HyoukJoong Lee, Jiquan Ngiam, Quoc Le, Yonghui Wu, and Zhifeng Chen. Gpipe: Efficient training of giant neural networks using pipeline parallelism. In *Advances in Neural Information Processing Systems*, 2019.
- [4] Zhihao Jia, Matei Zaharia, and Alex Aiken. Beyond data and model parallelism for deep neural networks. In *Proceedings of Machine Learning and Systems*, volume 1, pages 1–13, 2019.
- [5] Azalia Mirhoseini, Hieu Pham, Quoc V. Le, Benoit Steiner, Rasmus Larsen, Yuefeng Zhou, Naveen Kumar, Mohammad Norouzi, Samy Bengio, and Jeff Dean. Device placement optimization with reinforcement learning. In *Proceedings of the 34th International Conference on Machine Learning*, pages 2430–2439, 2017.
- [6] Deepak Narayanan, Aaron Harlap, Amar Phanishayee, Vivek Seshadri, Nikhil Devanur, Gregory R. Ganger, Phillip B. Gibbons, and Matei Zaharia. Pipedream: Generalized pipeline parallelism for DNN training. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles*, pages 1–15, 2019.
- [7] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

- [8] Samyam Rajbhandari, Olatunji Ruwase, Jeff Rasley, Yuxiong He, Ammar Ahmad Awan, and Michael Carbin. Zero: Memory optimizations toward training trillion parameter models. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–16, 2020.
- [9] Jie Ren, Samyam Rajbhandari, Reza Yazdani Aminabadi, Olatunji Ruwase, Shaden Yang, Minjia Li, and Yuxiong He. Zero-offload: Democratizing billion-scale model training. In *USENIX Annual Technical Conference*, pages 551–564, 2021.
- [10] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-LM: Training multi-billion parameter language models using model parallelism, 2019.
- [11] Lianmin Zheng, Zhihao Jia, Minmin Sun, Fei Wu, Chenguang Yu, Naser Hajinazar, Yida Wang, Jiayi Liu, Kuan Chen, Xiaoyu Zhu, et al. Alpa: Automating inter- and intra-operator parallelism for distributed deep learning. In *16th USENIX Symposium on Operating Systems Design and Implementation*, pages 559–578, 2022.